

[ETABLISSEMENT A RENSEIGNER]

[Filiere / specialite a renseigner]

Rapport de stage de fin d'études

SupportFlow

Plateforme intelligente de gestion, d'orchestration et de
supervision des tickets support

Etudiant(e) :	[Nom et prenom a renseigner]
Entreprise d'accueil :	[Entreprise d'accueil a renseigner]
Encadrant academique :	[Encadrant academique a renseigner]
Encadrant entreprise :	[Encadrant entreprise a renseigner]
Periode du stage :	[Periode de stage a renseigner]
Annee universitaire :	2025–2026

Document academique redige en LaTeX a partir du projet reel, de sa documentation, de son code source et de l'etat final de l'application.

Remerciements

Je tiens à adresser mes remerciements les plus sincères à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage et à l'aboutissement du projet SupportFlow.

Mes premiers remerciements s'adressent à l'équipe pédagogique de [Etablissement à renseigner], ainsi qu'aux enseignants de la filière [Filière / spécialité à renseigner], pour la qualité de la formation dispensée et pour les compétences techniques, méthodologiques et humaines qui m'ont permis d'aborder ce projet avec rigueur.

J'exprime également ma gratitude à mon encadrant académique, [Encadrant académique à renseigner], pour son suivi, ses conseils et son accompagnement tout au long du stage. Ses remarques ont été précieuses pour structurer ma réflexion, clarifier la démarche du projet et renforcer la qualité du livrable final.

Je remercie aussi mon encadrant au sein de [Entreprise d'accueil à renseigner], [Encadrant entreprise à renseigner], pour sa confiance, sa disponibilité et sa vision métier. Son accompagnement m'a aidé à mieux comprendre les enjeux concrets d'une plateforme de support, en particulier les questions de priorisation, de workflow, de supervision et de qualité de service.

Je souhaite également remercier l'ensemble des collaborateurs, collègues et personnes ressources qui ont facilité mon intégration et enrichi ce stage par leurs retours, leurs suggestions et leurs échanges. Les discussions autour des besoins utilisateurs, des choix techniques, de la sécurité, du BPMN, de la GED, de l'IA et du déploiement ont fortement contribué à la maturation de la solution.

Enfin, je remercie ma famille et mes proches pour leur soutien moral, leur patience et leurs encouragements tout au long de cette période exigeante.

Ce rapport constitue l'aboutissement d'un travail d'analyse, de conception, de réalisation, de correction et de validation. Il reflète non seulement le produit réalisé, mais aussi l'expérience humaine et professionnelle acquise durant ce stage.

Resume

Ce rapport presente le travail realise dans le cadre d'un stage de fin d'etudes consacre a la conception et a la consolidation de **SupportFlow**, une plateforme intelligente de gestion des tickets support. Le projet vise a repondre aux difficultes classiques rencontrees dans de nombreuses organisations : dispersion des demandes, manque de tracabilite, delais de traitement mal maitrises, visibilite insuffisante pour les managers, et faible capitalisation des solutions deja produites.

L'objectif principal etait de construire une application metier complete, capable d'orchestrer le cycle de vie d'un ticket depuis sa creation jusqu'a sa cloture et son archivage, tout en integrant des dimensions avancees telles que la gestion des habilitations, le suivi SLA, l'escalade, les notifications temps reel, l'archivage documentaire, le reporting et l'assistance intelligente.

La solution retenue repose sur une architecture full stack composee d'un frontend Angular, d'un backend Spring Boot, de Camunda pour le pilotage BPMN, de Keycloak pour l'authentification et l'autorisation, d'Alfresco pour la GED, ainsi que d'un agent IA base sur FastAPI et Ollama pour l'analyse et la suggestion. Le deploiement local et de demonstration est pris en charge par Docker Compose, tandis que la chaine GitOps s'appuie sur GitHub Actions, GHCR, Kubernetes, ArgoCD et SonarQube.

Au-dela de la simple realisation technique, le stage a permis de traiter des sujets transverses essentiels : clarification des roles client, agent et manager, securisation fine des actions sensibles, robustesse des scenarios de demonstration, reprise des workflows Camunda, validation des integrations, amelioration des temps de reponse et preparation d'une soutenance techniquement

defendable.

Le rapport met en lumière la problématique, la mission confiée, la démarche adoptée, les choix architecturaux, les résultats obtenus, les difficultés rencontrées ainsi que les perspectives d'évolution. Il montre que SupportFlow dépasse le cadre d'une application CRUD et se positionne comme un véritable système métier de support, démontrable, maintenable et évolutif.

Abstract

This internship report presents the design, implementation, stabilization and validation of **SupportFlow**, an intelligent support ticket management platform. The project addresses common support management issues such as scattered requests, weak traceability, poor SLA visibility, inconsistent supervision, and limited knowledge capitalization.

The main objective of the internship was to build and consolidate a complete business-oriented application able to manage the whole ticket lifecycle, from creation and qualification to assignment, treatment, client validation, escalation, reporting and archiving. The solution also integrates advanced concerns such as role-based access control, workflow orchestration, real-time notifications, document management and AI-assisted analysis.

The implemented platform combines an Angular frontend, a Spring Boot backend, Camunda BPM for workflow execution, Keycloak for identity and access management, Alfresco for document archiving, and a local AI assistant based on FastAPI and Ollama. For operational reproducibility, the project also includes a local Docker environment and a GitOps-oriented delivery chain built around GitHub Actions, GHCR, Kubernetes, ArgoCD and SonarQube.

Beyond coding tasks, the internship involved business analysis, architectural justification, integration troubleshooting, role consistency, workflow recovery, quality validation and demonstration readiness. The resulting platform is not limited to CRUD operations : it behaves as a structured support system with auditable processes, managerial supervision and operational value.

This report details the context of the internship, the business problem, the assigned mission,

the adopted methodology, the implemented solution, the validation results, the encountered difficulties and the proposed future improvements.

Table des matières

Remerciements	i
Resume	iii
Abstract	v
Glossaire	xx
1 Introduction generale	1
1.1 Contexte du stage	1
1.2 Presentation du sujet	1
1.3 Problematique	2
1.4 Objectifs du stage	3
1.5 Demarche generale adoptee	4
1.6 Interet academique et professionnel	4
1.7 Structure du rapport	5
2 Presentation de l'entreprise et cadre du stage	6
2.1 Presentation generale de l'entreprise d'accueil	6
2.1.1 Conseil de redaction	7

2.2	Equipe d'accueil et positionnement du projet	7
2.2.1	Synthese a personnaliser	8
2.3	Mission confiee	8
2.3.1	Formulation academique conseilee	9
2.4	Role de l'etudiant	9
2.4.1	Point de vigilance redactionnel	10
2.5	Cadre temporel et organisation du stage	10
2.6	Enjeux du stage	11
2.7	Remarque sur la personnalisation finale	11
3	Etude de l'existant et problematique	12
3.1	Contexte	12
3.2	Problematique	13
3.3	Justification du projet	13
3.4	Objectifs generaux	14
3.5	Resultat attendu	14
4	Analyse des besoins	15
4.1	Identification des acteurs	15
4.1.1	Le client	15
4.1.2	L'agent support	15
4.1.3	Le manager support	16
4.1.4	L'administrateur	16
4.2	Besoins fonctionnels	16
4.2.1	Gestion des tickets	16

4.2.2	Traitement et workflow	16
4.2.3	Communication et collaboration	17
4.2.4	Pilotage et supervision	17
4.2.5	Archivage et rapports	17
4.2.6	Aide intelligente	17
4.3	Besoins non fonctionnels	17
4.4	Cas d'usage majeurs	18
4.4.1	Cas d'usage 1 : creation d'un ticket	18
4.4.2	Cas d'usage 2 : qualification et assignation	19
4.4.3	Cas d'usage 3 : traitement par l'agent	19
4.4.4	Cas d'usage 4 : validation client	19
4.4.5	Cas d'usage 5 : escalade	19
4.5	Regles metier importantes	19
4.6	Valeur metier de la solution	20
5	Methodologie et conduite du projet	21
5.1	Approche generale	21
5.2	Organisation pratique du travail	21
5.3	Phases logiques de construction	22
5.3.1	Phase 1 : socle applicatif	22
5.3.2	Phase 2 : approfondissement metier	22
5.3.3	Phase 3 : automatisation et supervision	22
5.3.4	Phase 4 : experience avancee et IA	23
5.4	Planning synthetique du stage	23

5.5	Travail d'alignement	24
5.6	Gestion de la complexite	24
5.7	Role de la demonstration dans la methode	25
5.8	Difficultes rencontrees	25
5.9	Ajustements des objectifs	26
5.10	Lecons de conduite de projet	26
6	Conception generale de la solution	27
6.1	Vue d'ensemble architecturale	27
6.2	Justification des choix technologiques	27
6.2.1	Pourquoi Angular 17 ?	28
6.2.2	Pourquoi Spring Boot 3.2 ?	28
6.2.3	Pourquoi Camunda 7 ?	28
6.2.4	Pourquoi Keycloak ?	28
6.2.5	Pourquoi Docker Compose ?	29
6.3	Description des services Docker	29
6.4	Healthchecks et robustesse	30
6.5	Flux applicatifs principaux	30
6.6	Qualites architecturales obtenues	31
6.7	Modele de donnees et regles metier	31
6.8	Le ticket comme entite centrale	31
6.9	Autres entites structurantes	31
6.10	Attributs metiers importants	32
6.11	Statuts de ticket	32

6.12	Priorisation et SLA	33
6.13	Historique et auditabilite	33
6.14	Maturite metier observee	33
7	Realisation technique	34
7.1	Organisation en couches	34
7.2	Controllers recenses	34
7.3	Services metiers	35
7.4	TicketService comme coeur metier	36
7.5	EscalationService et aide a l'assignation	37
7.6	Services Camunda	37
7.7	Services documentaires et reporting	37
7.8	DTO, mapping et contrats API	38
7.9	Gestion des erreurs et robustesse	38
7.10	Observations globales sur le backend	38
7.11	Frontend et experience utilisateur	39
7.12	Role du frontend dans le projet	39
7.13	Structure frontend	39
7.14	Modules fonctionnels identifies	39
7.15	Services frontend	40
7.16	Experience utilisateur sur les tickets	40
7.17	Ecran assistant IA	41
7.18	Notifications et temps reel	41
7.19	Qualites et points d'attention du frontend	41

7.20	Securite, identite et autorisations	42
7.21	Keycloak comme socle IAM	42
7.22	Roles metiers	42
7.23	AuthorizationHelper	42
7.24	Coherence frontend-backend	43
7.25	Protection des clients	43
7.26	Autorisations sensibles	43
7.27	Role de la securite dans la soutenance	44
7.28	Workflow Camunda, SLA et automatisation	44
7.29	Pourquoi un workflow ?	44
7.30	Processus BPMN principal	44
7.31	Timers SLA	45
7.32	Valeur des timers	45
7.33	Synchronisation entre base et workflow	46
7.34	Camunda Cockpit comme outil de supervision	46
7.35	Processus et clarte metier	46
7.36	Limites et vigilance	46
7.37	Temps reel, notifications et collaboration	47
7.38	Importance du temps reel	47
7.39	WebSocket et STOMP	47
7.40	Role de NotificationService	47
7.41	Commentaires et pieces jointes	48
7.42	Alertes visibles dans l'interface	48

7.43	Portee collaborative du projet	48
7.44	Dimension IA et aide a la decision	48
7.45	Architecture IA	48
7.46	Cas d’usage IA	49
7.47	Interet concret	49
7.48	Gestion des lenteurs	49
7.49	Ergonomie et robustesse	50
7.50	Vision critique	50
7.51	Archivage, GED et reporting	50
7.52	Dimension documentaire du projet	50
7.53	Alfresco	50
7.54	Rapports PDF et Excel	51
7.55	Archivage dans le workflow	51
7.56	Valeur metier	51
7.57	Deploiement, exploitation et environnement de demonstration	51
7.58	Strategie de deploiement	52
7.59	Profils Docker	52
7.60	Ports et acces	52
7.61	Jeux de donnees et demonstrabilite	53
7.62	Importance de la coherence des seeds	53
7.63	Exploitation et diagnostic	53
8	Tests, validation et resultats	54
8.1	Importance des tests dans SupportFlow	54

8.2	Types de validation observables	54
8.3	Validation technique de la chaine qualite	55
8.4	GitHub Actions, SonarQube et GitOps	55
8.5	Validation des integrations externes	56
8.6	Corrections recentes significatives	56
8.6.1	Acceleration de la creation de ticket	56
8.6.2	Alignement des autorisations	56
8.6.3	Rehydratation des workflows Camunda	57
8.6.4	Modale d'assignation	57
8.6.5	Liste complete des candidats d'assignation	57
8.6.6	Outils IA ticket	57
8.6.7	Tri des clients	57
8.7	Enseignements tires de ces corrections	58
8.8	Validation des parcours par profil	58
8.8.1	Lecture par profil	58
8.9	Le client	58
8.9.1	Objectif du client	58
8.9.2	Parcours type	58
8.9.3	Attentes principales	59
8.10	L'agent support	59
8.10.1	Objectif de l'agent	59
8.10.2	Parcours type	59
8.10.3	Apports de l'IA	60

8.11 Le manager	60
8.11.1 Objectif du manager	60
8.11.2 Parcours type	60
8.11.3 Lecture de la modale d'assignation	61
8.12 L'administrateur	61
8.12.1 Objectif de l'administrateur	61
8.12.2 Vision systeme	61
8.13 Interet de cette lecture	62
8.14 Resultats observes	62
9 Analyse critique, apports et perspectives	63
9.1 Qualite logicielle	63
9.2 Maintenabilite	63
9.3 Analyse des risques techniques	64
9.4 Analyse des risques metiers	64
9.5 Observabilite et supervision	65
9.6 Qualite de demonstration	65
9.7 Maintenance evolutive	65
9.8 Apports techniques et professionnels	66
9.9 Analyse critique et perspectives	66
9.10 Forces du projet	66
9.11 Limites actuelles	67
9.12 Ameliorations metier recommandees	67
9.13 Ameliorations techniques recommandees	67

9.14 Vision d'évolution	68
10 Conclusion generale	69
Bibliographie et webographie	71
A Inventaire des controllers backend	73
B Inventaire des services backend	75
C Inventaire des services Docker	77
D Inventaire des modules frontend	78
E Matrice simplifiée des permissions	79
F Catalogue fonctionnel des endpoints metiers	80
G Scenarios de tests et d'acceptation	82
H Journal detaille de corrections recentes	85
H.1 Creation de ticket plus rapide	85
H.2 Resynchronisation Camunda	85
H.3 Clarification des autorisations	85
H.4 Modale d'assignation plus robuste	85
H.5 Affichage des candidats non eligibles	86
H.6 Resolution ID ou reference dans l'outil IA	86
H.7 Correction du tri des clients	86
H.8 Amelioration de la visibilite des alertes	86

H.9	Jeu de donnees de demonstration plus realiste	86
H.10	Lecon generale	87
I	Glossaire simplifie	88

Table des figures

6.1	Architecture generale de la solution SupportFlow	27
7.1	Processus ticket orchestre par Camunda	45
7.2	Chaine de build, qualite et deploiement de SupportFlow	52

Liste des tableaux

Glossaire

Terme	Definition
SLA	Service Level Agreement. Il s'agit des engagements de delai et de niveau de service associes au traitement d'un ticket.
Ticket	Dossier numerique representant une demande, un incident, une anomalie ou une action de support a traiter.
Workflow BPMN	Modele de processus metier orchestre par Camunda pour piloter les etapes du ticket.
Escalade	Mecanisme metier de requalification, de reassignation ou de supervision renforcee quand un ticket devient a risque ou bloque.
GED	Gestion electronique des documents. Dans SupportFlow, elle est assuree par Alfresco.
RBAC	Role-Based Access Control. Modele d'autorisation fonde sur les roles metier des utilisateurs.
GitOps	Mode de deploiement dans lequel l'etat cible de l'infrastructure et des applications est pilote par des manifests versionnes dans Git.
ArgoCD	Outil de synchronisation GitOps charge d'appliquer automatiquement les manifests Kubernetes.
SonarQube	Outil d'analyse statique utilise pour controler la qualite du code backend et frontend.

Ollama	Runtime local permettant d'exécuter des modèles IA pour le copilote SupportFlow.
Copilote IA	Ensemble de fonctions d'assistance : analyse de ticket, suggestion de réponse, résumé d'escalade et recommandations.
Takeover	Reprise forte d'un ticket par un niveau supérieur de supervision.

1 Introduction generale

1.1 Contexte du stage

La transformation numerique des organisations s'accompagne d'une augmentation continue des demandes de support, qu'elles concernent des incidents de production, des demandes d'accès, des anomalies applicatives, des questions fonctionnelles ou des besoins d'assistance lies a l'exploitation quotidienne des outils numeriques. Dans de nombreuses structures, ces demandes circulent encore par des canaux heterogenes comme l'email, la messagerie instantanee, le telephone ou des fichiers de suivi manuels. Cette dispersion rend la gestion du support difficile a piloter et fragilise la qualite de service.

Le stage s'inscrit dans ce contexte. Il porte sur la conception, la structuration et la stabilisation d'une solution appelee **SupportFlow**,ensee comme une plateforme unifiee de gestion des tickets. L'ambition du projet n'est pas seulement de centraliser les demandes, mais de construire un systeme metier capable d'orchestrer les flux de traitement, d'encadrer les roles, de surveiller les delais de resolution, d'archiver les preuves documentaires et d'assister les utilisateurs grace a l'intelligence artificielle.

1.2 Presentation du sujet

Le sujet consiste a mettre en place une application complete couvrant les besoins d'un centre de support moderne. L'application doit permettre a un client de creer un ticket, a un manager de

superviser la file de traitement, a un agent de prendre en charge et de resoudre les demandes, et a un administrateur de controler la plateforme. Cette logique multi-profil impose une gestion fine des autorisations, une lisibilite forte des etats metier et une architecture capable de faire communiquer plusieurs briques techniques.

SupportFlow articule ainsi plusieurs dimensions :

- une dimension **fonctionnelle**, car le ticket suit un cycle de vie metier clairement defini ;
- une dimension **organisationnelle**, car plusieurs acteurs cooperent selon des droits differencies ;
- une dimension **technique**, car le projet integre frontend, backend, moteur BPMN, GED, IAM, IA et deploiement ;
- une dimension **qualite**, car la plateforme doit etre demonstrable, testable, monitorable et exploitable.

1.3 Problematique

La problematique generale du stage peut etre formulee de la maniere suivante : *comment concevoir et consolider une plateforme de support capable de structurer le traitement des tickets, de garantir la securite des acces, d'automatiser les transitions critiques, d'ameliorer la supervision manageriale et d'integrer des fonctions d'assistance intelligente, tout en restant demonstrable et maintenable ?*

Cette problematique se decline en plusieurs questions plus precises :

- comment représenter correctement le cycle de vie d'un ticket et ses transitions sensibles ;
- comment faire converger les besoins du client, de l'agent, du manager et de l'administrateur dans une meme plateforme ;
- comment surveiller les delais SLA et detecter les situations a risque avant le depassement ;

- comment integrer un moteur de workflow et des notifications temps reel sans complexifier l’usage ;
- comment capitaliser les documents et les rapports produits autour des tickets ;
- comment introduire l’IA de facon utile, robuste et non bloquante pour le coeur metier.

1.4 Objectifs du stage

Le stage poursuit plusieurs objectifs complementaires. Le premier est de fournir une **solution applicative complete** couvrant les besoins essentiels du support. Le deuxieme est d’obtenir une **coherence metier** entre les ecrans, les roles, les APIs, les workflows et les regles d’autorisation. Le troisieme est de mettre en place une **chaîne technique credible** allant de la qualite logicielle au deploiement GitOps. Enfin, le quatrieme objectif est de produire un travail suffisamment mature pour etre defendu dans un cadre academique et professionnel.

De maniere plus concrete, le projet devait aboutir a :

- un portail ticketing exploitable par plusieurs profils ;
- un backend riche en logique metier et expose via des APIs REST coherentes ;
- une orchestration BPMN avec Camunda pour les etapes critiques du cycle ticket ;
- une authentication et des autorisations centralisees avec Keycloak ;
- un archivage documentaire via Alfresco ;
- des rapports PDF et Excel et des indicateurs de supervision ;
- une chaine de validation comprenant build, analyse qualite, scan securite et deploiement GitOps ;
- une documentation suffisante pour la soutenance et la reprise du projet.

1.5 Demarche generale adoptee

Le travail n'a pas été mené comme une simple succession de développements isolés. La démarche retenue a été itérative. Elle a consisté à construire un socle fonctionnel, à enrichir progressivement la profondeur métier, puis à corriger les incohérences révélées par les tests, les scénarios de démonstration et les retours d'usage. Ce mode de travail est particulièrement adapté à un projet comme SupportFlow, dans lequel chaque évolution touche plusieurs couches : interface, API, sécurité, workflow, données et services transverses.

La logique générale a donc été la suivante :

1. poser les briques structurantes du système ;
2. modéliser les acteurs, les besoins et les parcours principaux ;
3. mettre en place l'architecture technique et la base métier ;
4. enrichir la solution avec la supervision, l'IA, la GED et le reporting ;
5. corriger les points de friction les plus visibles ;
6. préparer une chaîne de preuve complète pour la maintenance.

1.6 Interet academique et professionnel

L'intérêt académique du projet est fort, car il mobilise de nombreuses compétences complémentaires : analyse des besoins, modélisation métier, développement full stack, sécurité applicative, orchestration de processus, qualité logicielle, conteneurisation, intégration continue, GitOps et assistance intelligente. Il permet aussi d'étudier la différence entre une application démonstrative simple et un système métier cohérent, capable de supporter des rôles, des délais, des traces, des documents et des scénarios réels.

D'un point de vue professionnel, SupportFlow répond à un besoin concret et récurrent. Une organisation qui maîtrise mal son support perd du temps, de la traçabilité, de la qualité de

service et de la capacite de capitalisation. Le projet traite donc un probleme reel, observable et economiquement important.

1.7 Structure du rapport

Le present rapport est organise en dix chapitres. Le premier introduit le contexte, le sujet, la problematique et la demarche generale. Le deuxieme presente l'entreprise d'accueil, le cadre du stage, la mission confiee et le role de l'etudiant. Le troisieme revient sur l'existant et reformule la problematique de maniere plus ciblee. Le quatrieme detaille l'analyse des besoins fonctionnels et non fonctionnels.

Le cinquieme chapitre expose la methodologie de conduite du projet, le planning, les difficultes rencontrees et les ajustements effectues. Le sixieme presente la conception generale de la solution, son architecture et son modele de donnees. Le septieme est consacre a la realisation technique. Le huitieme traite des tests, de la validation et des resultats observes. Le neuvieme propose une analyse critique des acquis, des limites et des perspectives. Enfin, le dixieme conclut le travail et synthetise la valeur globale du projet.

Des annexes viennent completer le corps principal avec des inventaires techniques, des scenarios de tests, des tableaux de permissions, des elements Docker et des details utiles a la reprise du projet sans alourdir excessivement la lecture principale.

2 Présentation de l'entreprise et cadre du stage

2.1 Présentation generale de l'entreprise d'accueil

Le stage a ete realise au sein de [Entreprise d'accueil a renseigner]. Dans la version finale de ce rapport, cette section doit presenter de maniere synthetique l'entreprise, son secteur d'activite, sa taille, son positionnement et son organisation generale. Lorsque certaines informations relevent de la confidentialite, il est possible de conserver une presentation sobre, centree sur la nature des activites et sur le contexte technologique utile a la comprehension du stage.

Element	Information a renseigner
Nom de l'entreprise	[Entreprise d'accueil a renseigner]
Secteur d'activite	[A completer]
Type de structure	[A completer]
Localisation	[A completer]
Service d'accueil	[A completer]
Nature du projet	Plateforme intelligente de gestion des tickets support

Pour une redaction academique complete, cette partie peut couvrir :

- le domaine d'activite principal de l'entreprise ;
- son type de clientele ou ses utilisateurs internes ;
- son organisation generale ou son departement systemes d'information ;
- les enjeux de qualite, de support ou de digitalisation dans lesquels s'inscrit le projet.

Dans le cas de SupportFlow, l'environnement d'accueil est pertinent car il confronte directement l'etudiant a un besoin transversal : mieux piloter les interactions entre utilisateurs, agents support, managers et outils techniques. Le projet ne nait donc pas d'une logique purement scolaire. Il prend sens dans un besoin d'industrialisation du support et de clarification des responsabilites.

2.1.1 Conseil de redaction

Pour finaliser cette section, il est recommande de partir d'un texte court en trois temps :

1. presenter l'entreprise et son activite principale ;
2. presenter le service ou l'equipe d'accueil ;
3. expliquer le lien direct entre cette equipe et le besoin SupportFlow.

2.2 Equipe d'accueil et positionnement du projet

Le projet s'inscrit generalement dans une equipe technique ou fonctionnelle liee a l'exploitation applicative, a la qualite de service, au support ou a la transformation digitale. Cette equipe constitue le point de contact entre les utilisateurs, les outils internes, les administrateurs et les responsables metier.

SupportFlow se positionne a l'intersection de plusieurs besoins :

- centraliser les demandes de support ;
- mieux qualifier les incidents et les demandes ;

- donner une visibilité claire au management ;
- rendre le traitement plus traçable et mesurable ;
- capitaliser les documents, rapports et résolutions ;
- préparer une trajectoire d'automatisation et d'assistance intelligente.

Ce positionnement explique la richesse du périmètre fonctionnel retenu. Le projet ne traite pas uniquement la saisie des tickets. Il couvre aussi le workflow, la sécurité, la priorisation, l'escalade, l'archivage, la notification et la qualité logicielle.

2.2.1 Synthèse à personnaliser

[A adapter] Le projet SupportFlow a été réalisé dans un contexte où la gestion des demandes de support devait gagner en traçabilité, en rapidité et en capacité de supervision. L'équipe d'accueil avait besoin d'un outil capable de centraliser les tickets, de clarifier les responsabilités, de structurer les délais de traitement et de produire des preuves de suivi exploitables.

2.3 Mission confiée

La mission confiée pendant le stage peut être formulée comme suit : *concevoir, réaliser, corriger et consolider une plateforme intelligente de support capable de piloter le cycle de vie des tickets de manière sécurisée, traçable et démontrable.*

Cette mission se déclinait en plusieurs sous-objectifs :

- analyser le besoin métier et identifier les acteurs du système ;
- définir l'architecture applicative et les technologies adéquates ;
- développer les couches frontend et backend ;
- intégrer Camunda, Keycloak, Alfresco, SonarQube et la chaîne GitOps ;

- mettre en place des parcours coherents pour le client, l'agent, le manager et l'administrateur ;
- assurer la validation du produit a travers des tests, des scenarios et des preuves techniques.

2.3.1 Formulation academique conseillée

Dans la version finale du rapport, la mission peut etre reformulee en un paragraphe continu de huit a douze lignes, en repondant a trois questions :

- quel probleme concret devait etre traite ;
- quelle solution etait attendue ;
- quelle a ete votre contribution personnelle dans cette solution.

2.4 Role de l'etudiant

Le role de l'etudiant dans ce stage a ete large et structurant. Il ne s'est pas limite a l'execution d'une tache ponctuelle. Il a combine plusieurs responsabilites complementaires :

- **role d'analyste**, pour reformuler le besoin, decouper les parcours et identifier les regles métier ;
- **role de concepteur**, pour choisir l'architecture, les composants et les integrations ;
- **role de developpeur full stack**, pour realiser le frontend Angular, le backend Spring Boot et les integrations associees ;
- **role d'integrateur**, pour faire converger Camunda, Keycloak, Alfresco, l'agent IA, Docker, SonarQube et GitHub Actions ;
- **role de testeur et de consolideur**, pour corriger les anomalies, stabiliser les parcours, verifier les permissions et fiabiliser la demonstration ;

- **role de redacteur technique**, pour documenter le produit, preparer la soutenance et structurer les preuves de fonctionnement.

Cette polyvalence constitue l'un des apports majeurs du stage. Elle montre que le travail accompli ne porte pas uniquement sur du code, mais aussi sur la coherence globale d'un systeme metier.

2.4.1 Point de vigilance redactionnel

Dans un rapport de stage, il est important de distinguer clairement :

- ce qui relevait de la mission globale du projet ;
- ce qui relevait de votre travail personnel ;
- ce qui relevait d'outils, de bibliotheques ou de briques deja existantes.

Cette distinction renforce la credibilite du rapport et aide le jury a comprendre la vraie valeur de votre contribution.

2.5 Cadre temporel et organisation du stage

Le stage s'est deroule sur la periode [Periode de stage a renseigner]. La conduite du projet a necessite une organisation rigoureuse, car plusieurs chantiers ont avance en parallele : realisation des fonctionnalites, correction des incoherences, gestion de la demonstration, stabilisation des environnements, mise en place de la qualite et preparation des livrables.

Le travail a suivi une logique progressive :

1. comprehension du besoin et cadrage du sujet ;
2. construction du socle technique et fonctionnel ;
3. enrichissement metier et integrations avancees ;

4. corrections transverses et mise en coherence des parcours ;
5. validation, documentation et preparation de la soutenance.

2.6 Enjeux du stage

Le stage presentait plusieurs enjeux. Le premier etait **metier** : il fallait produire une plateforme utile et lisible pour plusieurs profils. Le deuxieme etait **technique** : il fallait faire cooperer de nombreuses briques heterogenes sans perdre la robustesse de l'ensemble. Le troisieme etait **pedagogique** : il fallait montrer une maitrise reelle des concepts, des choix d'architecture et des validations. Enfin, le quatrieme etait **demonstratif** : le produit devait pouvoir etre presente de facon claire, avec des preuves tangibles de qualite, de securite et de deploiement.

En cela, SupportFlow constitue un sujet de stage complet. Il met l'etudiant dans une situation proche d'un projet professionnel reel, ou la qualite du resultat depend autant de la technique que de la capacite a clarifier le metier, stabiliser les integrations et documenter les choix.

2.7 Remarque sur la personnalisation finale

Cette version du rapport a volontairement conserve des placeholders afin de permettre une personnalisation finale rapide sans remettre en cause la structure academique du document. Avant depot final, il conviendra de remplacer les champs suivants :

- nom complet de l'etudiant ;
- nom de l'etablissement et de la specialite ;
- nom de l'entreprise d'accueil ;
- periode exacte du stage ;
- nom des encadrants academique et entreprise ;
- donnees non confidentielles permettant de presenter correctement le contexte d'accueil.

3 Etude de l'existant et problematique

3.1 Contexte

Dans une entreprise de services numeriques, de nombreuses demandes de support sont formulees chaque jour par des utilisateurs internes ou des clients externes. Ces demandes concernent des incidents de production, des anomalies applicatives, des demandes de correction, des demandes d'accès, des questions fonctionnelles ou des demandes de changements. Lorsque ce flux n'est pas structure, plusieurs difficultes apparaissent rapidement. Le support ne sait plus prioriser correctement, les utilisateurs ne savent pas ou en est leur dossier, les responsables n'ont pas de vision fiable des delais, et il devient presque impossible de capitaliser les solutions deja apportees.

Le projet SupportFlow s'inscrit dans ce contexte. Il cherche a proposer une solution complete de gestion des tickets, capable d'encadrer le cycle de vie d'une demande de support de maniere claire, tracable et mesurable. La presence d'un moteur BPMN, d'une GED et d'une couche IA montre que l'ambition du projet depasse la simple gestion d'un tableau de tickets. L'application se rapproche d'un systeme de support d'entreprise qui pourrait servir de base a un produit plus large.

3.2 Problematique

La problematique centrale peut etre formulee ainsi : comment concevoir une plateforme qui structure efficacement le traitement des tickets de support, garantissee la securite des acces, automatise les etapes critiques, surveille les engagements SLA et offre une experience claire a la fois aux agents, aux managers et aux clients ?

Cette problematique peut etre decomposee en plusieurs questions :

- Comment représenter le cycle de vie d'un ticket de façon explicite et évolutive ?
- Comment assurer un contrôle d'accès cohérent entre interface utilisateur, API backend et workflow ?
- Comment détecter les tickets à risque avant le dépassement d'un SLA ?
- Comment faciliter l'assignation et la réassignation des tickets en tenant compte des compétences et de la charge ?
- Comment conserver un historique fiable des actions et des décisions prises ?
- Comment enrichir le traitement du support avec des fonctions d'aide à la décision basées sur l'IA ?

3.3 Justification du projet

Le choix de réaliser un projet comme SupportFlow est pertinent pour plusieurs raisons. D'un point de vue pédagogique, il mobilise des compétences variées : backend Java, frontend Angular, sécurité OAuth2, modélisation BPMN, gestion documentaire, WebSocket, conteneurisation et intégration d'un composant IA. D'un point de vue métier, il met en scène un problème concret et récurrent dans les entreprises. D'un point de vue technique, il offre l'opportunité de travailler sur les sujets difficiles qui distinguent une application sérieuse d'une simple maquette : autorisations, workflow, performance perçue, reprise après incident, seed de données, supervision et

experience utilisateur.

3.4 Objectifs generaux

L'objectif principal de SupportFlow est de construire un socle fonctionnel permettant de gerer un ticket du debut a la fin, avec une separation claire des responsabilites. Cela suppose de fournir :

- une interface de creation et de consultation de tickets ;
- un moteur d'assignation et de prise en charge ;
- un suivi de statut et un historique d'actions ;
- une supervision des SLA et des alertes ;
- un systeme d'escalade ;
- un espace de validation client ;
- des fonctions d'archivage et de reporting ;
- un environnement de demonstration complet sous Docker ;
- un assistant IA pour acclereler certaines taches du support.

3.5 Resultat attendu

Le resultat attendu n'est pas seulement une application qui fonctionne en local, mais un demontreteur convaincant d'un produit support metier. En pratique, cela signifie que l'utilisateur doit percevoir un systeme coherent, dans lequel les boutons visibles correspondent a de vraies permissions, les statuts ont un sens metier clair, les workflows Camunda representent un comportement observable, et les donnees de test permettent de simuler des cas reels.

4 Analyse des besoins

4.1 Identification des acteurs

SupportFlow distingue quatre categories d'acteurs principales. Cette distinction structure tout le reste de l'application, depuis les routes frontend jusqu'aux controles backend.

4.1.1 Le client

Le client cree des tickets, consulte les tickets rattaches a son perimetre, lit les commentaires publics, ajoute des informations complementaires et valide ou refuse la resolution. Son role est volontairement borne : il doit pouvoir suivre l'etat de sa demande sans acceder a des informations internes au support.

4.1.2 L'agent support

L'agent traite les tickets qui lui sont assignes. Il peut analyser le probleme, ajouter des commentaires internes ou publics, joindre des pieces, prendre en charge un ticket, le resoudre et l'escalader si necessaire. Il est au coeur du traitement quotidien.

4.1.3 Le manager support

Le manager supervise la file de tickets. Il assigne les dossiers, arbitre les priorites, demande des revues manager, agit sur le SLA, gere les utilisateurs dans son perimetre et prend les decisions d'escalade. Son role est central pour la qualite de service.

4.1.4 L'administrateur

L'administrateur dispose d'un acces etendu. Il gere les utilisateurs, les clients, les configurations, les problemes de synchronisation et les actions exceptionnelles. C'est aussi lui qui intervient sur l'infrastructure et la maintenance de l'ecosysteme.

4.2 Besoins fonctionnels

Les besoins fonctionnels du systeme peuvent etre organises en sous-domaines.

4.2.1 Gestion des tickets

Le systeme doit permettre de creer, modifier, consulter, rechercher, filtrer et suivre des tickets. Chaque ticket doit porter des informations essentielles : reference, titre, description, client, categorie, priorite, severite, impact, agent assigne, statut, date de creation, date de mise a jour, SLA et historique.

4.2.2 Traitement et workflow

Le systeme doit rendre explicite le cycle de vie du ticket. Les etapes principales sont la qualification, l'assignation, la resolution, la validation client et l'archivage. Les transitions doivent etre controlees pour eviter des combinaisons incoherentes comme une assignation sur un ticket finalise ou une resolution sans agent responsable.

4.2.3 Communication et collaboration

Le systeme doit gerer des commentaires, des notifications et des pieces jointes. Les communications doivent distinguer ce qui est public et ce qui est interne. Les acteurs doivent etre informes en temps reel ou quasi temps reel des changements importants.

4.2.4 Pilotage et supervision

Le projet doit proposer un tableau de bord, des indicateurs de performance, une vision des tickets a risque, des alertes SLA et une capacite d'escalade. L'outil doit soutenir la decision manageriale.

4.2.5 Archivage et rapports

Une fois le ticket clos, le systeme doit etre capable de produire des rapports et d'archiver les elements utiles dans une GED. Cette dimension est importante pour l'audit, la conformite et la capitalisation.

4.2.6 Aide intelligente

Le systeme doit offrir une assistance intelligente qui n'interfere pas avec les transactions critiques. L'IA ne doit pas etre bloquante pour le fonctionnement normal, mais doit etre capable de fournir un gain d'efficacite lorsqu'elle est disponible.

4.3 Besoins non fonctionnels

Categorie	Exigence
-----------	----------

Performance	Les operations metier essentielles, notamment la creation de ticket et la consultation, doivent rester fluides meme lorsque des traitements annexes sont lents.
Securite	L'application doit authentifier les utilisateurs via Keycloak, proteger les endpoints et appliquer des autorisations metier fines.
Disponibilite	Le systeme doit pouvoir redemarrer ses services Docker et reconstituer un environnement coherent.
Tracabilite	Les actions importantes doivent laisser une trace exploitable.
Extensibilite	Les modules doivent etre assez decouples pour permettre l'ajout de nouvelles regles metier.
Demonstrabilite	Le projet doit pouvoir etre montre facilement en local avec un jeu de donnees significatif.
Maintenabilite	Le code doit etre structure en couches avec des services specialises.

4.4 Cas d'usage majeurs

4.4.1 Cas d'usage 1 : creation d'un ticket

Un utilisateur cree un ticket depuis l'interface. Il renseigne au minimum le client, le titre, la description et les informations necessaires a la priorisation. Le backend enregistre le ticket, calcule les attributs derives si necessaire, declenche le workflow Camunda, puis renvoie une reponse immediate au frontend. Des traitements annexes, comme les notifications ou certains declencheurs de workflow, peuvent etre deportes en asynchrone pour ne pas ralentir l'experience utilisateur.

4.4.2 Cas d’usage 2 : qualification et assignation

Le manager ouvre le ticket, l’évalue et decide quel agent doit en prendre la charge. L’interface d’assignation peut proposer une shortlist ou une liste complete avec etats d’eligibilite. Le backend verifie que l’action est autorisee et que le ticket est dans un etat compatible.

4.4.3 Cas d’usage 3 : traitement par l’agent

L’agent consulte les details du ticket, ajoute des commentaires, joint des documents, fait progresser l’investigation et soumet une resolution. Durant cette phase, le SLA doit etre surveille. Des alertes doivent remonter si le ticket devient a risque.

4.4.4 Cas d’usage 4 : validation client

Le client consulte la resolution. S’il l’accepte, le ticket peut progresser vers la cloture et l’archivage. S’il la rejette, le workflow doit revenir vers une etape de traitement.

4.4.5 Cas d’usage 5 : escalade

Lorsqu’un ticket devient critique, bloque ou depasse les delais, le systeme doit soit suggerer une escalation, soit la declencher selon les regles. L’historique doit permettre de comprendre pourquoi un ticket a ete reoriente.

4.5 Regles metier importantes

Les regles metier structurent le coeur du projet. Parmi les plus significatives :

- un ticket finalise ne doit plus pouvoir etre assigne ;
- un agent ne doit pas resoudre un ticket qui ne lui est pas attribue ;

- un client ne peut agir que sur son perimetre ;
- un manager peut superviser, reassigner et declencher certaines actions de controle ;
- le SLA doit etre surveille tout au long du traitement ;
- la validation client conditionne la cloture definitive du workflow ;
- les actions critiques doivent rester coherentes entre l’interface et le backend.

4.6 Valeur metier de la solution

SupportFlow apporte une valeur metier sur trois niveaux. Au niveau operationnel, il aide les equipes support a traiter les tickets avec plus de clarte. Au niveau manager, il rend visible la charge, les risques et les blocages. Au niveau client, il renforce la transparence et la confiance en montrant l’etat d’avancement du dossier. Ces trois niveaux font de SupportFlow un outil transverse qui aligne les objectifs du support, de l’encadrement et des utilisateurs finaux.

5 Methodologie et conduite du projet

5.1 Approche generale

La construction de SupportFlow s'apparente a une demarche iterative. Le projet mobilise plusieurs domaines difficiles a faire converger en une seule fois : developpement frontend, developpement backend, securite, workflow BPMN, temps reel, archivage documentaire et intelligence artificielle locale. Dans une telle configuration, la methode la plus realiste consiste a avancer par increments : construire un socle, l'enrichir, observer les effets de bord, puis consolider.

Cette methode iterative est visible a travers la documentation du depot, les rapports d'etat successifs, les scripts de diagnostic et les corrections recentes. Elle correspond a un mode de construction progressif dans lequel chaque nouvelle brique doit etre validee non seulement seule, mais aussi dans sa relation avec le reste de l'application.

5.2 Organisation pratique du travail

Sur le plan pratique, la conduite du projet a combine plusieurs activites menees en continu : analyse fonctionnelle, developpement frontend, developpement backend, integration des outils externes, tests manuels, tests scripts, corrections transverses et production documentaire. Cette organisation a impose un rythme de travail regulier, avec des allers-retours frequents entre implementation et verification.

Le projet n'a pas ete mene comme une suite de taches completement lineaires. Il a plutot evolue

selon un schema de boucles courtes :

1. identification d'un besoin ou d'un point de friction ;
2. mise en oeuvre d'une correction ou d'une nouvelle fonction ;
3. verification sur l'interface et sur les APIs ;
4. observation des effets de bord ;
5. documentation et consolidation.

5.3 Phases logiques de construction

Le projet peut etre relu selon une succession de grandes phases.

5.3.1 Phase 1 : socle applicatif

La premiere phase consiste a mettre en place le coeur de l'application : entites principales, CRUD de base, premiere interface, authentication et communication frontend-backend. Cette etape est indispensable pour etablir un squelette solide.

5.3.2 Phase 2 : approfondissement metier

Une fois le socle stable, le projet integre des dimensions plus riches : regles de priorisation, cycle de vie du ticket, commentaires, pieces jointes, tableaux de bord, notifications et roles differencies.

5.3.3 Phase 3 : automatisisation et supervision

L'ajout de Camunda et des mecanismes SLA transforme progressivement l'application en veritable outil de pilotage. Le ticket cesse d'etre un simple enregistrement pour devenir un processus pilotable.

5.3.4 Phase 4 : experience avancee et IA

La derniere grande phase introduit des briques a forte valeur ajoutee : GED, assistant IA, re-commandations d'assignation, copilote de ticket et scenarios de demonstration plus realistes.

5.4 Planning synthetique du stage

Le planning du stage peut etre relu en cinq grandes periodes. Cette presentation est volontairement synthetique ; elle pourra etre remplacee ou completee par un diagramme de Gantt en version finale.

Periode	Activites dominantes
Semaine 1 a 2	Cadrage du sujet, lecture du besoin, etude des briques techniques, prise en main du depot et de l'environnement de travail.
Semaine 3 a 5	Mise en place et consolidation du socle applicatif : entites principales, APIs, premiers ecrans, authentication et communication frontend-backend.
Semaine 6 a 8	Enrichissement metier : roles, priorites, SLA, notifications, commentaires, pieces jointes, detail ticket et supervision.
Semaine 9 a 11	Integrations avancees : Camunda, GED Alfresco, IA, reporting, chaine GitHub Actions, SonarQube et GitOps.
Semaine 12 et plus	Stabilisation, corrections transverses, tests complets, relecture documentaire, preparation de la soutenance et du rapport final.

5.5 Travail d'alignement

Un projet riche comme SupportFlow ne se juge pas seulement a la quantite de ses modules, mais a leur alignement. Les corrections recentes l'ont montre avec force. Il ne suffit pas que l'API accepte une action si le bouton correspondant n'est pas visible. Il ne suffit pas que le BPMN soit correct si le seed de donnees ne cree pas les bonnes instances. Il ne suffit pas que l'IA fonctionne si son delai degrade tout le parcours utilisateur.

La vraie valeur du travail recent tient justement dans cet alignement progressif :

- alignement entre permissions backend et boutons frontend ;
- alignement entre seed SQL et workflows Camunda ;
- alignement entre references visibles et identifiants internes ;
- alignement entre shortlist d'assignation et lecture metier des etats d'agents.

5.6 Gestion de la complexite

Chaque fonctionnalite importante met en jeu plusieurs couches. Par exemple, assigner un ticket peut impliquer une validation d'autorisation, une mise a jour en base, une notification, une evolution du workflow, une adaptation du detail ticket et parfois un recalcule d'indicateurs. Cette complexite n'est pas un signe de mauvaise conception ; elle est inherente a un outil metier complet.

L'enjeu consiste donc a rendre cette complexite gouvernable. SupportFlow y parvient par plusieurs moyens :

- separation controllers/services/repositories ;
- services specialises pour les domaines transverses ;
- documentation abondante ;

- scripts de tests et de diagnostic ;
- conteneurisation complete de l'environnement.

5.7 Role de la demonstration dans la methode

Une particularite forte du projet est l'importance accordee a la demonstration. Dans beaucoup de projets, l'environnement de demo est bricole en fin de parcours. Ici, il fait quasiment partie du produit. Les conteneurs, les seeds, Camunda Cockpit, Keycloak, Alfresco et meme l'agent IA composent un environnement complet qui peut etre montre.

Cette orientation influence la methode. Il faut penser non seulement a ce qui fonctionne en theorie, mais aussi a ce qui sera visible, comprehensible et reproductible en soutenance.

5.8 Difficultes rencontrees

Comme tout projet riche en integrations, SupportFlow a fait apparaitre plusieurs difficultes. Les principales ne venaient pas d'un unique composant, mais plutot des zones de jonction entre composants :

- incoherences ponctuelles entre les permissions backend et les boutons visibles dans le frontend ;
- ecart entre les donnees seed et l'etat reel attendu dans Camunda ;
- latences excessives lorsque des traitements annexes ou IA restaient dans le chemin critique ;
- incomprehensions metier possibles sur l'assignation et l'escalade ;
- sensibilite des scenarios de demonstration aux problemes d'environnement local.

Ces difficultes ont eu une valeur pedagogique importante. Elles ont montre qu'un projet applicatif complet se juge autant sur son integrabilite que sur ses fonctionnalites isolees.

5.9 Ajustements des objectifs

Au cours du stage, certains objectifs ont été ajustés pour privilégier la cohérence globale et la démontrabilité. Par exemple, l'effort n'a pas seulement porté sur l'ajout de nouvelles fonctions, mais aussi sur :

- la correction des points bloquants pour les profils client, agent et manager ;
- la clarification des droits métier ;
- la reprise de la chaîne GitOps pour obtenir une preuve de déploiement crédible ;
- l'industrialisation des éléments de soutenance : seeds, scripts, documentation, rapport et présentation.

Ce recentrage est logique dans un projet de fin d'études. Il vaut mieux produire une solution complète, cohérente et défendable qu'empiler des fonctions peu stabilisées.

5.10 Leçons de conduite de projet

Les leçons principales tirées de cette construction sont les suivantes :

- la première version fonctionnelle n'est jamais la version la plus révélatrice ;
- les frictions les plus importantes apparaissent souvent aux interfaces entre composants ;
- un bon seed de démonstration vaut presque autant qu'une bonne interface ;
- la centralisation des autorisations est indispensable dans un projet multi-profiles ;
- les composants lents comme l'IA doivent être traités avec des stratégies de fallback.

6 Conception generale de la solution

6.1 Vue d'ensemble architecturale

L'architecture de SupportFlow est de type full stack distribuee localement. Elle se compose d'un frontend Angular, d'un backend Spring Boot, d'une base MySQL, d'un moteur Camunda embarque dans le backend, d'un serveur Keycloak pour l'authentification, d'un ensemble Alfresco pour la GED, et d'un sous-systeme IA separe base sur FastAPI et Ollama.

Cette architecture est pertinente car elle decouple les responsabilites tout en restant facilement demonstrable. Chaque brique a un role clair. La separation entre frontend et backend est nette. La securite est deleguee a un composant dedie. Le workflow n'est pas code en dur dans une logique imperative opaque mais modelise en BPMN. La GED et l'IA sont integrees mais restent relativement independantes du coeur transactionnel.

Emplacement reserve pour le schema d'architecture globale de SupportFlow.
Insérer ici une figure finale montrant Angular, Spring Boot, Keycloak,
Camunda, Alfresco, l'agent IA, Docker et la chaine GitOps.

FIGURE 6.1 – Architecture generale de la solution SupportFlow

6.2 Justification des choix technologiques

6.2.1 Pourquoi Angular 17 ?

Angular est adapte a une interface d'entreprise riche, basee sur plusieurs ecrans, des tableaux, des formulaires complexes, des guards, des services HTTP et une gestion d'etat reactive. Angular Material facilite l'implementation rapide de composants standardises. Le projet exploite ce socle pour les tableaux pagines, les dialogues, les chips, les formulaires et les notifications visuelles.

6.2.2 Pourquoi Spring Boot 3.2 ?

Spring Boot permet de structurer rapidement une API REST robuste, securisee et modulable. L'ecosysteme Spring repond bien aux besoins du projet : web, validation, securite, data JPA, WebSocket, mail, Actuator. Le choix de Java 17 apporte un langage mature et bien adapte aux projets structurants.

6.2.3 Pourquoi Camunda 7 ?

Camunda repond a un besoin essentiel du projet : separer les transitions metier de la logique purement CRUD. Dans SupportFlow, le workflow du ticket est un objet metier en soi. Camunda permet d'en faire une representation visible, executable et instrumentable.

6.2.4 Pourquoi Keycloak ?

Un projet qui gere plusieurs roles et plusieurs types d'utilisateurs gagne beaucoup a externaliser l'authentification. Keycloak apporte un socle IAM moderne, des JWT standard, un realm configurable, une gestion des clients et un modele de roles souple.

6.2.5 Pourquoi Docker Compose ?

Docker Compose offre une excellente base pour un projet de soutenance. Il permet de figer l'environnement, de documenter les dependances, d'assurer la reproductibilite et de lancer facilement une demo complete.

6.3 Description des services Docker

Service	Image / origine	Port principal	Role
mysql	mysql:8.0	3306	Base de donnees principale.
keycloak	quay.io/keycloak /keycloak:23.0	8180	Authentification et gestion des roles.
backend	build local Spring Boot	8082 vers 8080	API REST, logique metier, moteur Camunda.
frontend	build Angular + Nginx	4200 vers 80	Interface utilisateur.
alfresco-postgres	postgres:14	interne	Base de donnees Alfresco.
alfresco	alfresco-content -repository-comm unity:7.4.0	8090	GED documentaire.
share	alfresco-share:7 .4.0	8091	Interface Alfresco Share.
sonarqube	sonarqube:commun ity	9000	Qualite de code, profil tools.

mailhog	mailhog/mailhog	8025 et 1025	SMTP de test, profil tools.
ollama	ollama/ollama:la test	11434	Serveur de modele local.
ollama-init	ollama/ollama:la test	interne	Telechargement du modele initial.
ai-agent	build FastAPI local	8000	Agent IA consommant Ollama.
demo-data-loader	mysql:8.0	interne	Recharge optionnelle des donnees de demo.

6.4 Healthchecks et robustesse

La presence de healthchecks dans le compose montre une preoccupation reelle pour la robustesse de l'environnement. Chaque composant critique expose une condition de sante simple mais efficace : ping MySQL, route de readiness Keycloak, endpoint Actuator backend, reponse HTTP frontend, verification de l'agent IA. Ce dispositif est utile en demonstration, mais aussi pour comprendre rapidement quel composant est en faute en cas de dysfonctionnement.

6.5 Flux applicatifs principaux

Le flux principal part du frontend, passe par le backend, mobilise la base, les services metiers et le moteur Camunda, puis renvoie l'information enrichie a l'utilisateur. Certains flux annexes se branchent en parallele :

- les notifications WebSocket repartent du backend vers le frontend ;
- les pieces jointes et rapports peuvent etre archives dans Alfresco ;
- les demandes d'assistance intelligente transitent du backend vers l'agent IA puis Ollama ;
- les actions d'authentification passent par Keycloak.

6.6 Qualites architecturales obtenues

L'architecture offre plusieurs qualites notables. D'abord, elle est pedagogiquement lisible : chaque composant a un role identifiable. Ensuite, elle est credibilisante : l'usage d'une IAM, d'un workflow BPMN et d'une GED rapproche le projet d'un SI d'entreprise. Enfin, elle est evolutive : il est possible d'enrichir les regles metier, de brancher d'autres sources documentaires ou de remplacer l'agent IA sans remettre en cause toute l'application.

6.7 Modele de donnees et regles metier

6.8 Le ticket comme entite centrale

Le ticket est l'objet principal de SupportFlow. Il concentre les dimensions fonctionnelles, techniques et organisationnelles du projet. A travers lui se croisent le client, l'agent, le manager, le SLA, le workflow, les notifications, les commentaires, les pieces jointes, la satisfaction et l'archivage.

Un bon modele de ticket doit donc porter plus que quelques champs textuels. Il doit pouvoir restituer l'etat courant du dossier, son urgence, son contexte, son historisation et les responsabilites associees.

6.9 Autres entites structurantes

Autour du ticket gravitent plusieurs entites structurantes :

- l'utilisateur ;
- le client ;
- le commentaire ;

- la notification ;
- la piece jointe ;
- l'historique de ticket ;
- les evenements d'escalade ;
- les elements de satisfaction.

6.10 Attributs metiers importants

Le projet mobilise plusieurs attributs metiers qui ne sont pas anecdotiques :

- priorite ;
- severite ;
- impact ;
- score ;
- agent assigne ;
- etat SLA ;
- niveau d'escalade ;
- date d'echeance ;
- date de creation ;
- date de mise a jour ;
- etat de validation client.

6.11 Statuts de ticket

Le projet a recours a plusieurs statuts. Ils sont essentiels pour piloter les transitions et conditionner les actions disponibles. Selon les parcours et les corrections recentes, les statuts ou

etats manipules couvrent des phases comme NEW, OPEN, IN_PROGRESS, PENDING, RESOLVED, CLOSED, ainsi que des etats lies a l'escalade ou au suivi SLA.

Un des enjeux du projet est justement de faire en sorte que chaque statut reste lisible du point de vue metier. Si plusieurs statuts se recouvrent ou si l'on ne sait plus quelle action est attendue, l'outil devient moins efficace.

6.12 Priorisation et SLA

La priorisation est fondee sur une combinaison de severite, d'impact et d'autres facteurs. Le but est de transformer une perception qualitative en un pilotage plus objectif. Le SLA ensuite traduit cette priorite en delai cible. Le moteur de workflow peut alors reactiver des alertes a des seuils precis.

6.13 Historique et auditabilite

Un projet de support serieux doit permettre de reconstituer l'histoire d'un ticket. Qui l'a cree ? Qui l'a assigne ? Quand a-t-il ete pris en charge ? Pourquoi a-t-il ete escalade ? Quel commentaire a provoque un changement d'orientation ? Le fait que SupportFlow embarque un historique dedie est donc une force structurante.

6.14 Maturite metier observee

Le modele de donnees n'est pas qu'un schema de persistance. Il porte une certaine maturite metier. On le voit dans la prise en compte du SLA, de la satisfaction, des pieces jointes, des historiques et des evenements d'escalade. Cela donne a l'application une profondeur superieure a celle d'un simple outil de tickets minimaliste.

7 Realisation technique

7.1 Organisation en couches

Le backend est structure autour d'une architecture en couches assez classique, mais bien adaptee au domaine. Les controllers exposent les endpoints. Les services encapsulent les regles metier. Les repositories accedent aux entites JPA. Les DTO protegent les contrats d'echange. Les mappers assurent les conversions. Cette structure simplifie la maintenance et facilite l'extension du projet.

7.2 Controllers recenses

Le backend contient seize controllers, chacun rattache a un sous-domaine metier ou technique.

- AgentWorkloadController
- AIAssistantController
- AttachmentController
- AuthController
- CamundaController
- ClientController
- CommentController
- DashboardController

- EscalationPolicyController
- KnowledgeBaseController
- NotificationController
- ReportController
- SatisfactionController
- SupportCategoryController
- TicketController
- UserController

Cette granularite montre que le domaine a ete decoupe de maniere raisonnable. Les responsabilites ne sont pas toutes concentrees dans un unique point d'entree.

7.3 Services metiers

Le backend recense vingt-quatre services principaux. Leur existence montre que les besoins ont ete separees par domaine fonctionnel et non traites dans des controllers trop lourds.

- AgentSkillService
- AgentWorkloadService
- AlfrescoArchiveService
- AlfrescoCmisService
- AttachmentService
- BusinessHoursService
- CamundaAsyncService
- CamundaBootstrapService
- CamundaService

- ClientService
- CommentService
- DashboardService
- EscalationService
- KeycloakAdminService
- KnowledgeBaseService
- NotificationService
- ReportService
- SatisfactionService
- SlaTimerService
- SupportCategoryService
- TicketAutomationService
- TicketService
- UserIdentityService
- UserService

7.4 TicketService comme coeur metier

Le service le plus central est naturellement TicketService. Il orchestre la creation, l’edition, l’assignation, la resolution, la cloture et certaines transitions associees au workflow. C’est aussi lui qui dialogue avec d’autres briques comme l’escalade, les notifications, le SLA et Camunda. L’une des qualites observees est que le service ne se contente pas de manipuler l’entite ticket de maniere naive. Il porte une veritable logique de garde-fous. Par exemple, certaines corrections recentes ont interdit l’assignation d’un ticket deja finalise. Ce type de regle est essentiel pour garantir l’integrite metier.

7.5 EscalationService et aide a l'assignation

EscalationService gere un aspect plus fin du domaine : la logique d'escalade et la production de listes de candidats a l'assignation. L'evolution recente du projet a permis de faire apparaitre non seulement les candidats eligibles, mais aussi les candidats non eligibles avec leur raison. Cette approche est tres utile d'un point de vue metier, car elle rend les decisions plus transparentes. Au lieu de faire disparaitre des profils de la liste, on montre qu'ils existent mais qu'ils sont indisponibles, deja assignes, en pause ou en capacite saturee.

7.6 Services Camunda

Le backend s'appuie sur plusieurs services Camunda. CamundaService assure l'integration principale. CamundaAsyncService permet de sortir du chemin critique certaines operations susceptibles de ralentir les reponses HTTP. CamundaBootstrapService joue un role particulier : il resynchronise ou rehydrate les workflows au demarrage, ce qui est tres utile lorsqu'un jeu de donnees est injecte directement en base et que les processus Camunda n'ont pas encore ete crees.

Cette derniere brique a une forte valeur pratique. Dans un contexte de demonstration, il est frequent de recharger des donnees de test sans repasser par toutes les API metier. Sans bootstrap de workflow, Camunda et la base divergeraient. La presence de ce mecanisme renforce donc la coherence globale du systeme.

7.7 Services documentaires et reporting

La couche documentaire est repartie entre trois services complementaires :

- AlfrescoCmisService pour la communication CMIS ;

- `AlfrescoArchiveService` pour la logique d’archivage ;
- `ReportService` pour la generation de rapports.

Cette separation est judicieuse. Elle evite de melanger logique documentaire, logique d’export et logique metier pure.

7.8 DTO, mapping et contrats API

Le projet emploie des DTO pour encapsuler les donnees exposees au frontend. Ce choix est pertinent pour plusieurs raisons. Il permet d’eviter d’exposer directement les entites JPA, de maitriser les champs transmis, de faire evoluer l’API plus sereinement et de ne pas coupler trop fortement la persistance avec la presentation. MapStruct est utilise pour certains mappages, ce qui reduit le bruit de code repetitif.

7.9 Gestion des erreurs et robustesse

Le backend contient un package `exception` et renvoie des erreurs semantiques. Au cours des corrections recentes, plusieurs erreurs visibles dans l’interface ont ete traitees pour devenir soit de vraies erreurs comprehensibles, soit des fallback plus robustes. L’exemple de l’outil IA sur ticket est representatif : auparavant, la saisie d’une reference de type 1013 pouvait etre comprise comme un ID inexistant et provoquer un 500. Desormais, le frontend est capable de resoudre une reference ou un ID avant d’appeler l’endpoint adequat.

7.10 Observations globales sur le backend

Le backend de SupportFlow donne l’image d’un projet qui a cherche a traiter de vrais sujets d’entreprise : workflow, SLA, reporting, GED, IAM, IA. Cette richesse implique inevitablement des zones de complexite, mais l’organisation generale du code reste lisible. Le point d’attention

principal est la synchronisation continue entre la logique metier, les autorisations et l'interface, ce qui fait justement partie des chantiers recents les plus importants.

7.11 Frontend et experience utilisateur

7.12 Role du frontend dans le projet

Le frontend Angular joue un role bien plus large qu'un simple client HTTP. Il constitue la vitrine du projet, l'espace de travail des utilisateurs et le lieu ou la coherence metier devient visible. Un bon backend ne suffit pas si l'interface laisse apparaitre des actions non autorisees, des informations contradictoires ou des etats difficiles a comprendre. L'une des forces de SupportFlow est d'avoir progressivement rapproche le frontend de la realite des permissions et des transitions metier.

7.13 Structure frontend

Le frontend se compose d'un socle core et d'un ensemble de modules features. Cette structure est saine. Le core regroupe les services transverses, les modeles, l'authentification et les acces aux APIs. Les features regroupent les ecrans utilisateurs selon le domaine metier.

7.14 Modules fonctionnels identifies

- dashboard
- tickets
- clients
- users
- profile

- `archives-reports`
- `ai-assistant`

7.15 Services frontend

Les services frontend jouent le role d'adaptateurs entre l'interface et l'API :

- `auth.service.ts`
- `ticket.service.ts`
- `client.service.ts`
- `user.service.ts`
- `dashboard.service.ts`
- `notification.service.ts`
- `report.service.ts`
- `ai.service.ts`
- `websocket.service.ts`

7.16 Experience utilisateur sur les tickets

Le domaine des tickets est naturellement le plus riche. La liste de tickets offre la vision d'ensemble. La fiche detail donne acces a l'ensemble des informations metier : statut, priorite, resume, SLA, actions, commentaires, pieces jointes, workflow et historique. L'ecran de creation concentre quant a lui les champs utiles a l'ouverture d'un dossier.

Une partie importante du travail recent a consiste a clarifier les actions disponibles sur la fiche ticket. Les boutons Assigner, Prendre le ticket, Resoudre, Escalader, Prolonger SLA, Revue manager, Archiver ou Reouvrir doivent tous reposer sur une logique coherente. Si

un agent non autorise voit un bouton actif qui lui sera ensuite refuse par le backend, l'outil perd en credibilite. Le recent alignement entre frontend et backend sur les autorisations est donc un progres majeur.

7.17 Ecran assistant IA

L'ecran AI Assistant montre l'ambition du projet. Il propose un chat, des outils par ticket et une analyse globale des tendances. Cet ecran centralise plusieurs types d'assistance : analyse, diagnostic, suggestion de reponse et resume d'escalade.

Une correction recente a apporte une amelioration tres utile : l'utilisateur peut maintenant saisir soit un ID interne, soit une reference comme SF-1013, soit meme le suffixe numerique 1013. L'interface resout ce format en amont avant de contacter l'API IA. Cela evite les erreurs inutiles et rapproche l'outil du langage naturel des utilisateurs.

7.18 Notifications et temps reel

Le frontend exploite un service WebSocket pour recevoir des notifications. Cette capacite renforce l'impression d'un vrai poste de supervision. Les alertes, notifs et badges visibles dans l'interface contribuent a rendre le systeme plus vivant. L'utilisateur ne travaille pas sur un simple ecran statique, mais sur une application qui reactualise ses informations.

7.19 Qualites et points d'attention du frontend

Le frontend offre deja une bonne couverture fonctionnelle et une identite visuelle forte. Les points de vigilance concernent surtout la clarte continue des parcours complexes, la gestion des erreurs, la prevention des ecrans de chargement trop longs et la coherence totale entre les informations affichees et les regles metier reelles. Les derniers correctifs montrent que ce sujet

est bien pris en compte.

7.20 Securite, identite et autorisations

7.21 Keycloak comme socle IAM

Le choix de Keycloak apporte une architecture de securite moderne. Le frontend delegue l'authentification au serveur IAM. Le backend verifie ensuite les jetons et en extrait les roles. Cette separation est saine car elle evite de reinventer une partie tres sensible du systeme.

7.22 Roles metiers

SupportFlow distingue quatre roles metiers principaux :

- ADMIN
- SUPPORT_MANAGER
- SUPPORT_AGENT
- CLIENT

Cette matrice n'est pas purement cosmetique. Elle structure les vues, les actions et les donnees accessibles.

7.23 AuthorizationHelper

Le fichier AuthorizationHelper illustre bien l'effort recent de centralisation des permissions. Il ne se contente pas de savoir si un utilisateur est manager ou agent. Il verifie aussi des permissions d'action contextualisees : assigner, prendre en charge, resoudre, escalader, gerer le SLA, demander une revue manager, cloturer, rejeter une resolution ou archiver.

Cette approche est plus robuste qu'une simple logique de role brut. Elle combine role, identite courante, etat du ticket et rattachement effectif du ticket a l'utilisateur. C'est exactement ce qu'il faut pour un outil metier dans lequel une meme ressource ne doit pas etre traitable par tous de la meme facon.

7.24 Coherence frontend-backend

L'un des problemes les plus frequents dans les applications d'entreprise est l'ecart entre les autorisations calculees dans l'interface et celles appliquees cote serveur. SupportFlow a connu une phase de ce type, ce qui est normal dans un projet en croissance. Le recent travail d'alignement a justement vise a faire disparaitre les raccourcis trop permissifs dans le frontend, notamment la confusion entre staff et manager.

7.25 Protection des clients

Le role client merite une attention particuliere. Il doit pouvoir acceder a ses tickets sans obtenir une visibilite excessive sur les donnees du support. Les verifications de proprietes de ticket et de rattachement au client sont donc essentielles. Elles renforcent la separation entre espace client et espace interne.

7.26 Autorisations sensibles

Parmi les actions les plus sensibles :

- l'assignation ;
- la reassignation ;
- la resolution ;
- la cloture ;

- l’escalade ;
- la gestion du SLA ;
- l’archivage ;
- l’accès aux commentaires internes.

Le fait que ces actions soient maintenant contrôlées plus explicitement constitue une nette amélioration de la qualité globale du projet.

7.27 Role de la securite dans la soutenance

Dans une soutenance ou un rapport de stage, la sécurité est souvent survolée. Or, dans SupportFlow, elle constitue un véritable axe de profondeur. Le projet montre que la gestion des rôles ne sert pas seulement à cacher ou afficher des menus, mais qu’elle conditionne la validité métier des actions possibles.

7.28 Workflow Camunda, SLA et automatisations

7.29 Pourquoi un workflow ?

Le workflow est au cœur de l’identité du projet. Dans beaucoup d’applications de tickets, le cycle de vie est codé par une succession de if et de mises à jour de statut. Cela fonctionne jusqu’à un certain point, mais devient difficile à maintenir et à expliquer. Avec Camunda, SupportFlow rend visible le processus. Cela apporte une dimension de pilotage et de démonstration très forte.

7.30 Processus BPMN principal

Le processus observé dans le fichier BPMN suit une logique simple et convaincante :

1. creation du ticket ;
2. qualification par le manager ;
3. resolution par l'agent ;
4. validation par le client ;
5. decision de cloture ou retour en traitement ;
6. archivage ;
7. fin du processus.

Ce schema repond bien au besoin d'un support structure. Il rend claire la place de chaque acteur.

Emplacement reserve pour un schema BPMN ou une capture Camunda Cockpit.
La version finale peut illustrer la creation du ticket, la qualification manager, la resolution agent, la validation client et l'archivage.

FIGURE 7.1 – Processus ticket orchestre par Camunda

7.31 Timers SLA

Le workflow integre des boundary events sur l'etape de resolution. Cela signifie que le temps n'est pas un simple attribut calcule en arriere-plan; il devient un element de comportement du processus. Les seuils a 50%, 75% et 100% permettent de distinguer un suivi simple, une situation a risque et un depassement caracterise.

7.32 Valeur des timers

Cette mecanique est importante d'un point de vue metier. Attendre le depassement du SLA pour agir est trop tardif. Un bon systeme doit donner des signaux faibles avant la rupture. Le niveau a risque joue donc un role preventif, tandis que le niveau de depassement active des mesures plus fortes.

7.33 Synchronisation entre base et workflow

Un point souvent sous-estimé dans les projets BPM concerne la synchronisation entre l'état persiste et l'état du moteur. Dans SupportFlow, ce sujet est traité de manière explicite grâce au bootstrap Camunda. Lorsqu'un jeu de données est injecté directement, le backend peut recréer les instances ou les faire progresser proprement pour que Cockpit retrouve un état pertinent. Cette capacité a une valeur forte en démonstration et en exploitation.

7.34 Camunda Cockpit comme outil de supervision

L'existence de Cockpit offre une seconde lecture du système. Le ticket peut être observé depuis l'interface métier, mais aussi depuis le moteur de workflow. Cela permet de vérifier visuellement qu'un ticket se trouve bien à l'étape attendue, qu'une tâche humaine existe ou qu'une instance a été créée.

7.35 Processus et clarté métier

L'un des apports majeurs du BPMN est de clarifier ce qui est attendu à chaque étape. Un ticket non assigné reste en qualification. Un ticket pris en charge entre dans une résolution active. Un ticket valide déclenche l'archivage. Cette clarté est précieuse autant pour les développeurs que pour les utilisateurs métier.

7.36 Limites et vigilance

Le workflow ne doit pas devenir un doublon confus de la logique applicative. Toute l'intelligence du projet consiste à maintenir une bonne articulation entre états métier, processus BPMN et comportements du frontend. Les corrections récentes montrent que ce travail d'alignement

reste un point central.

7.37 Temps reel, notifications et collaboration

7.38 Importance du temps reel

Dans un systeme de support, l'information perd de sa valeur si elle arrive trop tard. Les notifications temps reel rendent l'application plus reactive. Elles reduisent le besoin de recharger les pages, renforcent la perception d'activite et facilitent la coordination entre agents, managers et clients.

7.39 WebSocket et STOMP

Le backend s'appuie sur Spring WebSocket et le frontend sur STOMP. Ce duo est classique et efficace pour des notifications applicatives. Il convient bien a un projet de supervision comme SupportFlow, ou plusieurs types d'evenements doivent etre diffuses : creation, assignation, changement de statut, commentaire, alerte SLA et escalade.

7.40 Role de NotificationService

NotificationService n'est pas un simple mecanisme de toast visuel. Il formalise les evenements importants du domaine. Cela permet de donner une existence explicite aux alertes, de compter certaines notifs, de les afficher dans des badges et de garder une trace de leur emission.

7.41 Commentaires et pieces jointes

La collaboration ne passe pas seulement par les statuts. Les commentaires permettent de contextualiser une decision, d'expliquer un blocage, de demander des informations ou de documenter une resolution. Les pieces jointes enrichissent le dossier avec des captures, journaux, exports ou documents metier.

7.42 Alertes visibles dans l'interface

Les corrections recentes ont aussi porte sur la maniere de rendre visibles les alertes SLA et les notifications sur la page de detail ticket. Les badges rouges et les compteurs ne sont pas uniquement decoratifs ; ils servent a attirer l'attention sur ce qui exige une action.

7.43 Portee collaborative du projet

Ces mecanismes renforcent le caractere collaboratif de l'application. Le ticket devient un point de convergence entre plusieurs personnes et non un objet isole entre un createur et un traitant.

7.44 Dimension IA et aide a la decision

7.45 Architecture IA

Le composant IA est volontairement heberge dans un service separe. Le dossier ai-agent repose sur FastAPI, Uvicorn, HTTPX et la bibliotheque ollama. Le backend principal consomme ensuite cet agent. Cette separation a plusieurs avantages : elle isole les dependances IA, permet d'ajuster la politique de timeout et evite de polluer le backend metier avec des considerations de modele.

7.46 Cas d'usage IA

L'IA intervient sur plusieurs parcours :

- analyse d'un ticket ;
- diagnostic ;
- suggestion de reponse ;
- copilote sur le detail ticket ;
- resume d'escalade ;
- tendances ;
- recommandation d'assignation.

7.47 Interet concret

L'interet de cette brique n'est pas de remplacer l'agent, mais de lui faire gagner du temps. Une suggestion de reponse peut accelerer la redaction d'un retour client. Un diagnostic peut orienter l'investigation. Un resume d'escalade peut faire gagner du temps lors d'un changement d'intervenant. Un copilote peut condenser le contexte d'un dossier long.

7.48 Gestion des lenteurs

L'IA locale peut etre lente, surtout sur une machine de developpement ou avec un modele CPU-only. SupportFlow a justement connu ce type de contrainte. Les derniers correctifs ont donc cherche a reduire l'impact percu de cette lenteur :

- timeout court sur certaines routes critiques comme la recommandation d'assignation ;
- fallback immediate sur des listes de secours ;
- deplacement de certaines operations hors du chemin critique de creation.

7.49 Ergonomie et robustesse

Une autre amelioration importante a consiste a accepter plusieurs formes d'identification de ticket dans l'outil IA : identifiant interne, reference complete ou suffixe numerique. Ce petit detail a une grande valeur ergonomique. Il rapproche l'interface des habitudes reelles des utilisateurs, qui manipulent souvent les references visibles plutot que les IDs de base de donnees.

7.50 Vision critique

L'IA constitue une valeur ajoutée forte, mais elle ne doit jamais devenir un point de fragilité du système principal. Le découplage retenu dans SupportFlow va dans le bon sens. A l'avenir, il serait possible de renforcer la capitalisation de la base de connaissances et la traçabilité des suggestions IA retenues ou rejetées.

7.51 Archivage, GED et reporting

7.52 Dimension documentaire du projet

Un ticket de support est souvent accompagné de nombreuses traces : captures d'écran, logs, mails, exports, rapports, justificatifs d'action. L'archivage est donc une dimension essentielle si l'on veut dépasser une logique de simple ticketing temporaire.

7.53 Alfresco

L'intégration Alfresco montre que SupportFlow ne traite pas le document comme une pièce jointe secondaire. L'usage d'une GED permet de mieux structurer la conservation et la consultation de certains contenus, notamment dans une perspective d'audit ou de capitalisation.

7.54 Rapports PDF et Excel

Le projet reference explicitement des services de generation PDF et Excel. Cela ouvre plusieurs usages :

- extraction managere ;
- justification d'un traitement ;
- partage hors application ;
- archive formelle du dossier.

7.55 Archivage dans le workflow

L'etape `archive_ticket` du processus BPMN montre que l'archivage n'est pas considere comme un detail optionnel, mais comme une vraie etape de fin de cycle. Cette integrite de conception est un point fort du projet.

7.56 Valeur metier

Le reporting et la GED donnent au projet une dimension plus enterprise. Ils permettent de montrer qu'un ticket resolu ne disparaît pas simplement d'une liste, mais peut alimenter une memoire documentaire exploitable.

7.57 Deploiement, exploitation et environnement de demonstration

7.58 Strategie de deployment

Le projet est pense pour etre deploye localement via Docker Compose. Cette strategie repond parfaitement aux besoins d'un PFE ou d'une soutenance. Elle simplifie le lancement, minimise les dependances manuelles et rend l'environnement plus portable.

Emplacement reserve pour la chaine CI/CD et GitOps.
La figure finale peut montrer GitHub Actions, GHCR, SonarQube, Kubernetes,
ArgoCD et les composants SupportFlow.

FIGURE 7.2 – Chaine de build, qualite et deployment de SupportFlow

7.59 Profils Docker

Le fichier de compose distingue des services toujours actifs et d'autres lies a des profils comme tools ou demo. C'est un bon compromis entre richesse fonctionnelle et maitrise des ressources.

7.60 Ports et acces

Le systeme expose plusieurs points d'accès utiles pour la demonstration :

- frontend sur le port 4200 ;
- backend sur le port 8082 ;
- Keycloak sur le port 8180 ;
- Alfresco sur 8090 ;
- Share sur 8091 ;
- SonarQube sur 9000 ;
- agent IA sur 8000 ;
- Ollama sur 11434 ;
- MailHog sur 8025.

7.61 Jeux de donnees et demonstrabilite

L'existence de `data-dev.sql`, `data-test.sql` et `data-docker.sql` montre que la demonstration a ete prise au serieux. Le jeu de donnees Docker a notamment ete enrichi pour offrir des cas plus realistes, avec plusieurs clients, plusieurs tickets, plusieurs notifications et des situations de criticite differenciees.

7.62 Importance de la coherence des seeds

L'un des enseignements du projet est qu'un seed de base ne suffit pas. Si l'application fait intervenir un moteur BPMN, une simple insertion SQL ne garantit pas la coherence globale. Il faut aussi penser a la creation des workflows ou a leur rehydratation. La correction apportee via le bootstrap Camunda va exactement dans ce sens.

7.63 Exploitation et diagnostic

Le depot contient de nombreux scripts de test et de diagnostic pour Keycloak, Camunda, RBAC ou les parcours de demonstration. Cette richesse documentaire est un atout fort pour la maintenance et la soutenance.

8 Tests, validation et resultats

8.1 Importance des tests dans SupportFlow

Etant donne la richesse fonctionnelle du projet, les tests ne peuvent pas se limiter a un simple ping des endpoints. Il faut valider la coherence metier, l'integrite des permissions, la synchronisation du workflow, le comportement de l'interface et la solidite du seed de demonstration.

8.2 Types de validation observables

Dans le depot, on retrouve plusieurs formes de validation :

- scripts de test PowerShell ;
- scripts de demonstration shell ;
- healthchecks Docker ;
- compilation Maven ;
- build Angular ;
- verifications manuelles par parcours utilisateur ;
- observation de Camunda Cockpit.

8.3 Validation technique de la chaine qualite

La validation du projet ne s'est pas limitee aux parcours fonctionnels. Une partie importante du travail a consiste a verifier que la chaine technique etait elle aussi exploitable. Cette verification s'appuie sur plusieurs preuves :

- compilation backend Maven ;
- build frontend Angular en mode production ;
- analyse SonarQube sur le backend Java et le frontend TypeScript ;
- scan securite des images conteneur avec Trivy ;
- construction et publication des images applicatives dans GHCR ;
- mise a jour des manifests Kubernetes et synchronisation GitOps via ArgoCD.

8.4 GitHub Actions, SonarQube et GitOps

Le projet SupportFlow dispose d'une chaine CI/CD structuree. Dans le depot, le workflow GitHub Actions couvre la compilation, le build frontend, l'analyse qualite, le scan securite et la preparation du deploiement. Cette chaine joue un double role : elle valide le code et elle constitue une preuve concrete de maturite technique pour la soutenance.

L'analyse SonarQube permet d'evaluer la qualite du code de facon reproductible. Dans le projet, elle est mobilisee comme preuve de bonne pratique, non comme simple accessoire. De meme, la partie GitOps montre que le deploiement cible ne repose pas uniquement sur une execution manuelle locale, mais sur une logique versionnee et synchronisable.

8.5 Validation des integrations externes

Une plateforme comme SupportFlow n'est correcte que si ses integrations essentielles repondent ensemble. Les validations ont donc porte sur :

- Keycloak pour l'authentification et les roles ;
- Camunda pour les processus et timers SLA ;
- Alfresco pour l'archivage et la consultation documentaire ;
- l'agent IA pour l'analyse, la suggestion et le resume d'escalade ;
- Kubernetes et ArgoCD pour la preuve de synchronisation GitOps.

8.6 Corrections recentes significatives

Le projet a connu plusieurs corrections importantes qui illustrent une phase de consolidation tres interessante.

8.6.1 Accelération de la creation de ticket

La creation de ticket pouvait paraitre lente car certains traitements annexes bloquaient la reponse. Le travail recent a consiste a deplacer certains traitements Camunda et certaines notifications hors du chemin critique de la creation. Ce type de correction montre une bonne comprehension de la difference entre logique metier synchrone et taches asynchrones.

8.6.2 Alignement des autorisations

Des incoherences existaient entre ce que le frontend laissait voir et ce que le backend autorisait reellement. Le recent durcissement de la matrice RBAC cote backend et son alignement cote frontend ont apporte une meilleure clarte.

8.6.3 Rehydratation des workflows Camunda

Les tickets injectes directement en base ne creaient pas automatiquement leurs processus Camunda. La mise en place d'un service de bootstrap a corrige ce decalage et a restaure la lisibilite du moteur dans Cockpit.

8.6.4 Modale d'assignation

L'ecran de validation manager de l'assignation pouvait rester charge trop longtemps. Le projet a ete corrige pour charger d'abord une liste de secours, puis enrichir avec l'IA si elle repond. Ce comportement est bien plus robuste.

8.6.5 Liste complete des candidats d'assignation

La modale d'assignation n'affichait initialement qu'une shortlist restreinte. Elle montre desormais aussi les profils non eligibles avec leur etat. Cette modification augmente fortement la comprehensibilite du systeme.

8.6.6 Outils IA ticket

L'assistant IA pouvait retourner des erreurs lorsque l'utilisateur saisissait une reference de ticket au lieu de l'ID interne. La resolution automatique des identifiants a corrige ce point et rendu l'interface plus naturelle.

8.6.7 Tri des clients

Une erreur de tri sur le champ name au lieu de companyName provoquait un 500. Ce type de correction est discret mais important, car il montre l'importance de l'alignement entre frontend et modele backend.

8.7 Enseignements tires de ces corrections

Ces corrections montrent trois choses. D'abord, les vrais problemes se situent souvent a l'interface entre composants, plus que dans les composants eux-memes. Ensuite, les parcours utilisateurs et les seeds de demonstration jouent un role majeur dans la qualite percue. Enfin, le projet gagne en maturite a mesure qu'il transforme les erreurs brutes en comportements robustes et comprehensibles.

8.8 Validation des parcours par profil

8.8.1 Lecture par profil

Pour apprecier la qualite fonctionnelle de SupportFlow, il est utile de lire l'application non pas uniquement par modules techniques, mais par profils utilisateurs. Cette lecture montre comment l'outil se comporte du point de vue des besoins concrets.

8.9 Le client

8.9.1 Objectif du client

Le client souhaite avant tout declarer un probleme, suivre son traitement et obtenir une resolution claire. Il ne cherche pas a gerer la complexite interne du support; il attend surtout transparence et reactivite.

8.9.2 Parcours type

Le parcours type du client est le suivant :

1. connexion ;
2. creation d'un ticket ;
3. ajout de precisions si necessaire ;
4. lecture des reponses publiques ;
5. suivi de l'etat ;
6. validation ou refus de la resolution.

8.9.3 Attentes principales

Les attentes principales du client sont simples mais exigeantes :

- savoir si sa demande a bien ete prise en compte ;
- comprendre l'etat du traitement ;
- disposer d'un retour lisible ;
- pouvoir refuser une resolution insuffisante.

8.10 L'agent support

8.10.1 Objectif de l'agent

L'agent a besoin d'un espace de travail clair lui permettant de comprendre rapidement le dossier, d'agir sans ambiguite et de laisser des traces utiles pour la suite. Il doit sentir que l'outil l'aide reellement plutot qu'il ne lui impose une charge administrative inutile.

8.10.2 Parcours type

Le parcours type de l'agent comprend :

- consulter sa file ;

- prendre un ticket si necessaire ;
- analyser le probleme ;
- commenter ;
- joindre des preuves ou documents ;
- produire une resolution ;
- eventuellement escalader.

8.10.3 Apports de l'IA

Pour l'agent, l'IA n'a de valeur que si elle accelere des gestes repetitifs. Une suggestion de reponse ou un resume de contexte peuvent aider, mais l'outil doit rester pleinement utilisable meme si l'IA est indisponible.

8.11 Le manager

8.11.1 Objectif du manager

Le manager cherche a piloter. Son besoin principal n'est pas de traiter chaque ticket a la place de l'agent, mais de voir les zones de tension : surcharge, tickets sans proprietaire, tickets a risque, depassements, retours client, demandes d'arbitrage.

8.11.2 Parcours type

Le manager ouvre les tableaux de bord, consulte les listes, attribue les dossiers, suit les alertes SLA, demande des revues manager et intervient lorsque l'organisation doit etre reajustee.

8.11.3 Lecture de la modale d'assignation

L'évolution de la modale d'assignation est très représentative du besoin manager. Une simple shortlist opaque ne suffit pas. Le manager a besoin de comprendre pourquoi certains agents sont proposés et pourquoi d'autres ne le sont pas. L'affichage des états déjà assignés, en pause, occupé ou capacité atteinte répond bien à cette attente.

8.12 L'administrateur

8.12.1 Objectif de l'administrateur

L'administrateur agit comme garant de l'intégrité globale. Il intervient sur les utilisateurs, les clients, l'infrastructure, les redémarrages, les seeds de demo et les problèmes de synchronisation. Son espace de travail est donc hybride : à la fois métier et technique.

8.12.2 Vision système

Pour l'administrateur, la valeur de SupportFlow tient aussi dans les outils annexes :

- Camunda Cockpit ;
- Keycloak ;
- Alfresco Share ;
- healthchecks Docker ;
- scripts de diagnostic.

8.13 Interet de cette lecture

La lecture par profil montre que SupportFlow n'est pas pense comme une simple interface uniforme pour tous. Chaque acteur y trouve un usage specifique, des ecrans differents et des responsabilites propres. Cette approche renforce la pertinence metier du projet.

8.14 Resultats observes

Les validations menees permettent de conclure que le projet atteint un niveau de maturite convaincant pour un stage de fin d'etudes. Les parcours critiques sont demonstrables, les integrations majeures sont visibles, les APIs principales sont exploitables et la chaine de qualite est documentee. Les resultats les plus importants sont les suivants :

- un cycle ticket coherent du client jusqu'a l'archivage ;
- une supervision manager plus claire grace aux files de priorite, aux alertes et aux escalades ;
- une experience agent amelioree par les outils d'aide et les actions directes ;
- une preuve de qualite logicielle et de deploiement au travers de GitHub Actions, SonarQube et GitOps ;
- une base documentaire et une soutenance mieux preparees grace a la formalisation des guides et des preuves.

9 Analyse critique, apports et perspectives

9.1 Qualite logicielle

La qualite logicielle de SupportFlow se lit a plusieurs niveaux. Au niveau du code, le decoupage en couches et en services specialises constitue deja un socle solide. Au niveau fonctionnel, la qualite se mesure a la coherence des parcours. Au niveau systeme, elle depend de la capacite a relancer l'environnement complet et a retrouver un etat credible de demonstration.

9.2 Maintenabilite

Le projet est relativement maintenable grace a :

- une architecture modulaire ;
- une documentation abondante ;
- un environnement Docker reproductible ;
- des scripts de test et de diagnostic ;
- des conventions metier de plus en plus explicites.

La maintenance doit cependant rester vigilante sur les points de jonction entre les composants. Une regression sur un champ de tri, sur une permission ou sur un identifiant de ticket peut

provoquer des erreurs visibles importantes meme si le coeur technique du systeme reste sain.

9.3 Analyse des risques techniques

Risque	Impact	Reponse possible
Lenteur de l'IA locale	Mauvaise experience utilisateur	Timeout court, fallback et asynchronisme.
Desynchronisation base / Camunda	Incoherence de supervision	Bootstrap Camunda et verification Cockpit.
Permissions incoherentes	Refus ou exces d'accès	Centralisation des autorisations.
Seed trop pauvre ou errone	Demo peu representative	Jeu de donnees realiste et controlee.
Erreur de mapping frontend / backend	Erreurs serveur evitables	Alignement strict des contrats et champs.
Traitements lourds dans le chemin critique	Latence ressentie	Deporter les taches lentes.
Regression multi-role	Parcours casses pour certains profils	Tests role par role.

9.4 Analyse des risques metiers

Les risques metiers sont parfois plus graves qu'un bug purement technique. Un manager qui ne comprend pas pourquoi un agent n'apparaît pas dans une liste, un client qui ne comprend pas

pourquoi son ticket reste bloqué, ou un agent qui voit une action qu'il ne peut pas exécuter sont autant de situations qui diminuent la confiance dans l'outil.

9.5 Observabilité et supervision

L'observabilité est bien servie par le projet :

- Camunda Cockpit permet de vérifier les instances et les tâches ;
- les healthchecks Docker indiquent l'état des composants ;
- les logs aident au diagnostic ;
- les scripts de test facilitent la vérification des parcours.

9.6 Qualité de démonstration

Dans un contexte de soutenance, la qualité de démonstration est un critère de succès. Support-Flow répond bien à ce besoin car il ne demande pas seulement d'expliquer des concepts ; il permet de les montrer. On peut ouvrir le frontend, observer Keycloak, vérifier les workflows dans Cockpit, consulter la GED, voir les notifications et relancer l'ensemble via Docker.

9.7 Maintenance évolutive

La maintenance évolutive pourrait s'orienter vers :

- une supervision manager encore plus orientée files d'attente ;
- des règles métier plus explicites sur les tickets en attente ;
- une base de connaissances plus structurée ;
- des tests end-to-end plus nombreux ;
- une observabilité plus approfondie des traitements asynchrones.

9.8 Apports techniques et professionnels

Au niveau technique, le projet a permis de consolider plusieurs acquis majeurs : conception d'APIs REST, découpage frontend/backend, modelisation des roles et permissions, orchestration BPMN, utilisation d'une GED, deploiement conteneurise, integration continue et demarche GitOps. L'etudiant a egalement acquis une meilleure maitrise des strategies de diagnostic, notamment lorsqu'un probleme ne vient pas d'un composant unique mais de leur interaction.

Au niveau professionnel, ce stage a renforce la capacite a raisonner en termes de valeur metier, de priorisation, de soutenabilite et de preuve de fonctionnement. Il a aussi developpe des reflexes importants : documenter, verifier, rejouer les scenarios critiques, preparer des jeux de donnees coherents et ne pas confondre livraison de fonctionnalites et stabilisation globale du produit.

9.9 Analyse critique et perspectives

9.10 Forces du projet

SupportFlow presente plusieurs forces evidententes :

- une architecture complete et credibilisante ;
- une bonne profondeur fonctionnelle ;
- une vraie integration de BPM, IAM, GED et IA ;
- un effort de demonstrabilite remarquable ;
- une progression recente vers plus de clarte metier.

9.11 Limites actuelles

Le projet reste toutefois exigeant a stabiliser complètement. Plus le perimetre est large, plus les points de jonction sont nombreux. Les principaux points d'attention concernent :

- la maitrise de tous les parcours role par role ;
- la precision des statuts metier et de leurs transitions ;
- la dependance a une IA locale potentiellement lente ;
- la necessite d'un plus grand nombre de tests de non-regression automatisees.

9.12 Ameliorations metier recommandees

- structurer explicitement le concept de `waitingOn` pour savoir si un ticket attend le client, l'agent, le manager ou un tiers ;
- imposer des motifs obligatoires pour les pauses SLA, prolongations et escalades ;
- enrichir la supervision manager avec une vue file d'attente, charge et risque ;
- renforcer la base de connaissances issue des resolutions validees ;
- rendre encore plus visibles la prochaine action attendue et la raison de l'etat courant.

9.13 Ameliorations techniques recommandees

- etendre les tests d'integration role par role ;
- renforcer la gestion centralisee des erreurs API ;
- instrumenter davantage la supervision des traitements asynchrones ;
- ajouter des tests de parcours end-to-end navigateur sur les scenarios critiques ;
- formaliser un contrat d'API de capacites de ticket pour les actions disponibles.

9.14 Vision d'évolution

SupportFlow peut évoluer de plusieurs façons. Il peut devenir un outil interne de support IT, un démonstrateur commercial, ou un socle d'application de gestion d'incidents plus large. La présence d'une GED, d'un moteur BPMN et d'une IA locale ouvre aussi des perspectives de capitalisation, de reporting avancé et d'automatisation progressive.

10 Conclusion generale

Ce stage a eu pour objectif de concevoir, realiser et consolider une plateforme de gestion intelligente des tickets support capable de repondre a des besoins metier reels : centralisation des demandes, clarification des responsabilites, supervision des delais, archivage documentaire, assistance a la decision et preparation d'une demonstration techniquement defendable.

Le projet SupportFlow montre qu'une application de support ne peut pas etre reduite a une simple logique de creation et de cloture de tickets. Pour etre utile, elle doit prendre en compte les acteurs, les regles metier, les contraintes de securite, le suivi SLA, la capitalisation documentaire et les exigences d'observabilite. C'est pourquoi la solution proposee s'appuie sur une architecture riche, combinant Angular, Spring Boot, Camunda, Keycloak, Alfresco, Docker, SonarQube, GitHub Actions, ArgoCD et une brique IA locale.

Au fil du stage, plusieurs dimensions ont ete traitees : l'analyse du besoin, la conception de l'architecture, la realisation technique, la gestion des roles et des permissions, la structuration des parcours client, agent et manager, la reprise des workflows Camunda, la stabilisation de la chaine GitOps, l'integration de la GED, la generation des rapports et la mise en place de preuves qualite. Cette progression a permis de faire evoluer SupportFlow d'un ensemble de fonctionnalites vers un systeme metier plus coherent, plus explicable et plus solide.

Les resultats obtenus sont significatifs. L'application permet aujourd'hui de gerer des tickets selon un cycle de vie structure, d'exposer des parcours differencies selon les roles, de surveiller les tickets a risque, de produire des rapports, d'archiver les documents, d'utiliser un copilote IA et de s'inscrire dans une demarche DevOps et GitOps documentee. Le projet ne se contente

donc pas de fonctionner localement : il dispose aussi d'un socle de qualite, de deploiement et de maintenance.

Sur le plan personnel, ce stage a ete formateur a plusieurs niveaux. Il a permis de renforcer les competences techniques en developpement full stack, securite, BPMN, conteneurisation et integration continue. Il a aussi developpe des competences methodologiques essentielles : analyse du besoin, priorisation, gestion des risques, correction transverse, documentation et preparation d'une demonstration convaincante.

Bien entendu, le projet n'est pas fige. Plusieurs perspectives existent : renforcer encore les tests de non-regression, enrichir la base de connaissances, pousser plus loin les vues manager de supervision, rendre l'IA plus observable et industrialiser davantage le deploiement multi-environnements. Ces limites ne diminuent pas la valeur du travail realise ; elles montrent au contraire que SupportFlow constitue une base serieuse pour des evolutions futures.

En conclusion, le stage a permis de repondre de maniere satisfaisante a la problematique initiale. SupportFlow apporte une solution credible, integree et evolutive pour la gestion des tickets support. Il illustre la capacite a conduire un projet complexe de bout en bout, depuis le besoin metier jusqu'a la preuve technique, tout en produisant un resultat coherent pour l'entreprise et pertinent dans un cadre academique.

Bibliographie et webographie

Bibliographie

- [1] Institution de rattachement, *Rapport de stage - note pedagogique*, document de reference remis pour l'encadrement du memoire de stage.
- [2] Spring, *Spring Boot Reference Documentation*, documentation officielle de Spring Boot.
- [3] Angular, *Angular Documentation*, documentation officielle du framework Angular.
- [4] Camunda, *Camunda 7 Documentation*, documentation officielle du moteur BPMN Camunda.
- [5] Keycloak, *Server Administration Guide*, documentation officielle de Keycloak.
- [6] Alfresco, *Alfresco Content Services Documentation*, documentation officielle de la GED Alfresco.
- [7] SonarSource, *SonarQube Documentation*, documentation officielle relative a l'analyse statique et a la qualite logicielle.
- [8] Argo Project, *Argo CD Documentation*, documentation officielle relative a la synchronisation GitOps.
- [9] Kubernetes, *Kubernetes Documentation*, documentation officielle du socle d'orchestration conteneurisee.
- [10] Ollama, *Ollama Documentation*, documentation officielle du runtime d'execution locale des modeles IA.
- [11] Projet SupportFlow, *Documentation interne du depot : README, guides fonctionnels, guides DevOps/GitOps, guide SonarQube et support de soutenance.*

A Inventaire des controllers backend

Controller	Responsabilite principale
AgentWorkloadController	Charge de travail des agents et vues de supervision.
AIAssistantController	Endpoints d'analyse, diagnostic, suggestion, copilote et tendances.
AttachmentController	Upload, recuperation et suppression des pieces jointes.
AuthController	Authentification locale ou aides de developpement selon les profils.
CamundaController	Consultation ou pilotage d'informations liees au moteur BPM.
ClientController	CRUD et recherche sur les clients.
CommentController	Gestion des commentaires publics et internes.
DashboardController	KPIs et statistiques.
EscalationPolicyController	Regles et configuration d'escalade.
KnowledgeBaseController	Base de connaissances.
NotificationController	Notifications utilisateur.
ReportController	Export et rapports.

SatisfactionControlle Sondages et indicateurs de satisfaction.

r

SupportCategoryContro Categorie de support et referentiel associe.

ller

TicketController Coeur des actions ticket.

UserController Gestion des utilisateurs.

B Inventaire des services backend

Service	Responsabilite principale
AgentSkillService	Gestion des competences agents.
AgentWorkloadService	Calcul de charge et supervision des agents.
AlfrescoArchiveService	Archivage metier vers la GED.
AlfrescoCmisService	Communication CMIS avec Alfresco.
AttachmentService	Gestion des fichiers joints.
BusinessHoursService	Regles d'horaires ouvrables et calculs associes.
CamundaAsyncService	Traitements Camunda asynchrones.
CamundaBootstrapService	Resynchronisation des processus au demarrage.
CamundaService	Integration principale avec le moteur BPMN.
ClientService	Logique metier client.
CommentService	Commentaires et regles associees.
DashboardService	KPI et statistiques.
EscalationService	Escalade et shortlist d'assignation.
KeycloakAdminService	Interactions d'administration avec Keycloak.
KnowledgeBaseService	Capitalisation des solutions.
NotificationService	Generation et diffusion des notifications.
ReportService	Rapports PDF et Excel.
SatisfactionService	Gestion de la satisfaction client.

SlaTimerService	Reaction aux seuils de SLA.
SupportCategoryService	Gestion des categories de support.
TicketAutomationService	Automatisation de taches autour du ticket.
e	
TicketService	Cycle de vie principal des tickets.
UserIdentityService	Resolution de l'identite courante.
UserService	Gestion metier des utilisateurs.

C Inventaire des services Docker

Service	Profil	Port	Observation
mysql	standard	3306	Base principale avec healthcheck.
keycloak	standard	8180	IAM du systeme.
backend	standard	8082	API et Camunda integres.
frontend	standard	4200	UI Angular servie par Nginx.
alfresco-postgres	standard	interne	DB du referentiel documentaire.
alfresco	standard	8090	Repository documentaire.
share	standard	8091	UI Alfresco Share.
sonarqube	tools	9000	Analyse qualite optionnelle.
mailhog	tools	8025/1025	Serveur SMTP de test.
ollama	standard	11434	Runtime des modeles locaux.
ollama-init	standard	interne	Pull du modele au demarrage.
ai-agent	standard	8000	FastAPI pour les fonctions IA.
demo-data-loader	demo	interne	Rechargement controle du seed de demo.

D Inventaire des modules frontend

Module frontend	Description
dashboard	Vue de pilotage avec KPI, tendances, repartitions et performance.
tickets	Liste, detail, creation, edition, assignation, escalade, workflow et historique.
clients	Gestion des clients, recherche, detail et actions associees.
users	Gestion des utilisateurs et de leurs profils.
profile	Espace personnel de l'utilisateur connecte.
archives-reports	Consultation et exploitation des archives et rapports.
ai-assistant	Chat IA, diagnostic, suggestion, analyse ticket et tendances.
layout/header	Barre haute, badges de notifications, menu utilisateur.
layout/sidebar	Navigaton laterale adaptee aux roles.
core/services	Services HTTP et fonctions transverses.

E Matrice simplifiee des permissions

Action	Client	Agent	Manager	Admin
Creer un ticket	Oui	Oui	Oui	Oui
Voir ses tickets	Oui	N/A	N/A	N/A
Voir tickets assignes	Non	Oui	Oui	Oui
Voir tous les tickets	Non	Non	Oui	Oui
Assigner un ticket	Non	Non	Oui	Oui
Prendre le ticket	Non	Oui si	Oui	Oui
		autorise		
Resoudre un ticket	Non	Oui si assigne	Oui selon contexte	Oui selon contexte
Escalader	Non	Oui si assigne	Oui	Oui
Gerer le SLA	Non	Oui si assigne	Oui	Oui
Demander une revue manager	Non	Non	Oui	Oui
Cloturer	Oui selon parcours	Non	Oui	Oui
Archiver	Non	Non	Oui selon regle	Oui
Voir commentaires internes	Non	Oui	Oui	Oui
Gerer utilisateurs	Non	Non	Oui selon ecrans	Oui
Gerer clients	Non	Non	Oui	Oui

F Catalogue fonctionnel des endpoints

metiers

Methode	Endpoint	Usage
GET	/api/tickets	Liste paginee des tickets.
GET	/api/tickets/search	Recherche multicritere.
GET	/api/tickets/{id}	Detail d'un ticket.
GET	/api/tickets/reference/{ref}	Recuperation par reference.
POST	/api/tickets	Creation d'un ticket.
PUT	/api/tickets/{id}	Mise a jour d'un ticket.
POST	/api/tickets/{id}/assign/{agentId}	Assignment.
POST	/api/tickets/{id}/take-charge	Prise en charge.
POST	/api/tickets/{id}/resolve	Resolution.
POST	/api/tickets/{id}/close	Cloture.
POST	/api/tickets/{id}/archive	Archivage.
POST	/api/tickets/{id}/escalate	Escalade manuelle.
POST	/api/tickets/{id}/manager-review	Revue manager.
POST	/api/tickets/{id}/sla-pause	Pause SLA.
POST	/api/tickets/{id}/sla-resume	Reprise SLA.
POST	/api/tickets/{id}/sla-extend	Extension SLA.
POST	/api/tickets/{id}/reject-resolution	Refus de resolution.

GET	/api/tickets/{id}/comments	Commentaires complets.
GET	/api/tickets/{id}/comments/public	Commentaires publics.
POST	/api/tickets/{id}/comments	Ajout de commentaire.
GET	/api/tickets/{id}/attachments	Liste des pieces jointes.
POST	/api/tickets/{id}/attachments	Upload d'une piece jointe.
GET	/api/tickets/{id}/history	Historique du ticket.
GET	/api/tickets/{id}/workflow-status	Etat du workflow.
GET	/api/tickets/{id}/workflow-trace	Trace du workflow.
GET	/api/clients	Liste des clients.
GET	/api/users	Liste des utilisateurs.
GET	/api/dashboard/*	KPI et statistiques.
GET	/api/notifications/*	Recuperation des notifications.
GET	/api/ai/status	Etat de la brique IA.
GET	/api/ai/analyze/{id}	Analyse IA d'un ticket.
GET	/api/ai/diagnose/{id}	Diagnostic IA.
GET	/api/ai/suggest-response/{id}	Suggestion de reponse.
GET	/api/ai/escalation-summary/{id}	Resume d'escalade.
POST	/api/ai/chat	Chat contextualise.

G Scenarios de tests et d'acceptation

ID	Scenario	Resultat attendu
T1	Creation d'un ticket par un client	Le ticket est cree, reference generee, workflow lance.
T2	Creation d'un ticket sans client cote staff	Le systeme bloque si la regle client obligatoire s'applique.
T3	Assignment par manager	Le ticket change d'etat et l'agent est notifie.
T4	Assignment d'un ticket finalise	Action refusee.
T5	Prise en charge par l'agent assigne	Le ticket passe dans un etat compatible.
T6	Resolution par un agent non assigne	Action refusee.
T7	Escalade manuelle par agent assigne	Historique et destinataire d'escalade mis a jour.
T8	Revue manager sur ticket non finalise	Action autorisee aux bons profils.
T9	Pause SLA sur ticket actif	Etat SLA passe en pause avec tracabilite.
T10	Reprise SLA	Chronometrage repris correctement.

T11	Ticket a risque a 75%	Alerte visible cote interface et notification envoyee.
T12	Ticket depasse a 100%	Processus d'escalade ou supervision declenche.
T13	Validation client de la resolution	Workflow progresse vers archivage.
T14	Refus client de la resolution	Retour en traitement ou en qualification selon le flux.
T15	Consultation Camunda Cockpit	Les instances visibles correspondent aux tickets actifs.
T16	Upload de piece jointe	Le fichier apparait dans le ticket.
T17	Archivage documentaire	Rapport et dossier archives accessibles.
T18	Consultation des notifications temps reel	Les badges et compteurs se mettent a jour.
T19	Utilisation de l'assistant IA avec reference ticket	La reference est resolue sans erreur technique.
T20	Tri des clients par raison sociale	La liste charge sans erreur serveur.
T21	Chargement de la modale d'assignation	Une liste s'affiche meme si l'IA est lente.
T22	Affichage des candidats non eligibles	Les badges expliquent pourquoi certains agents ne sont pas selectionnables.
T23	Rechargement du seed Docker	La base et Camunda restent coherents.
T24	Connexion manager	Les boutons manager sont visibles et fonctionnels.

T25	Connexion agent	Les actions de supervision non autorisees ne sont pas proposees.
T26	Connexion client	Le client ne voit que son perimetre.
T27	Build backend	La compilation Maven doit aboutir.
T28	Build frontend	Le build Angular doit aboutir.
T29	Docker healthchecks	Les conteneurs critiques doivent passer en healthy.
T30	Parcours complet incident critique	De la creation a l'archivage, toutes les etapes restent coherentes.

H Journal detaille de corrections recentes

H.1 Creation de ticket plus rapide

Le chemin de creation du ticket a ete revu pour eviter que des traitements secondaires retardent trop la reponse visible par l'utilisateur. Cette correction ameliore nettement la sensation de fluidite.

H.2 Resynchronisation Camunda

L'initialisation de workflows pour des donnees semees directement en base a ete prise en charge afin de rendre le moteur BPM de nouveau coherent avec les tickets existants.

H.3 Clarification des autorisations

Les autorisations ont ete revues pour que le frontend et le backend racontent la meme chose. Cette clarification ameliore la credibilite de l'outil.

H.4 Modale d'assignation plus robuste

La modale charge maintenant une liste de secours avant de tenter un enrichissement IA, ce qui evite les ecrans de chargement infinis.

H.5 Affichage des candidats non éligibles

L'utilisateur voit désormais pourquoi certains profils ne sont pas sélectionnables. Cette transparence est précieuse pour la compréhension métier.

H.6 Résolution ID ou référence dans l'outil IA

L'assistant IA accepte désormais un identifiant interne ou une référence visible comme SF-1013. Cela rapproche l'outil du langage réel des utilisateurs.

H.7 Correction du tri des clients

Le frontend envoie maintenant un champ de tri compatible avec le backend, ce qui évite une erreur serveur évitable.

H.8 Amélioration de la visibilité des alertes

Des compteurs et badges reliés à de vraies données ont été ajoutés pour mettre en évidence les actions requises et les alertes sur les tickets.

H.9 Jeu de données de démonstration plus réaliste

La base a été rechargée avec des données plus riches afin de mieux illustrer les cas réels de support, de supervision et d'escalade.

H.10 Lecon generale

La plupart de ces corrections ne changent pas l'idée du produit, mais elles changent fortement la qualité perçue. Cela montre qu'un bon projet se joue aussi dans les détails de cohérence.

I Glossaire simplifie

Terme	Definition
Ticket	Dossier representant une demande de support ou un incident.
SLA	Engagement de delai de traitement ou de resolution.
Escalade	Reorientation ou supervision renforcee d'un ticket.
Camunda	Moteur BPMN charge d'executer le workflow metier.
Keycloak	Serveur d'identite et d'authentification.
Alfresco	GED utilisee pour l'archivage documentaire.
Ollama	Runtime local pour les modeles de langage.
FastAPI	Framework Python utilise pour l'agent IA.
RBAC	Controle d'accès basé sur les rôles.
Workflow	Enchainement structure des etapes du traitement.
Cockpit	Interface Camunda de supervision des processus.
Shortlist	Liste reduite de candidats proposes pour une assignation.